

```

import pandas as pd

# A tiny hand-made dataset to illustrate the concepts above
student_data = {
    "RollNo": [1, 2, 3, 4, 5],
    "Name": ["Aarav", "Diya", "Kabir", "Meera", "Ishan"],
    "Age": [19, 20, 19, 21, 20],
    "Gender": ["M", "F", "M", "F", "M"],
    "CGPA": [8.4, 9.1, 7.8, 8.9, 7.2],
    "Placed": ["Yes", "Yes", "No", "Yes", "No"],
}

student_df = pd.DataFrame(student_data)
student_df  # displaying the DataFrame

{"summary": "{\n  \"name\": \"student_df\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"RollNo\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1,\n        \"min\": 1,\n        \"max\": 5,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          2,\n          5,\n          3\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"Diya\",\n          \"Ishan\",\n          \"Kabir\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Age\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 19,\n        \"max\": 21,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          19,\n          20,\n          21\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Gender\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"F\",\n          \"M\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"CGPA\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.7854934754662193,\n        \"min\": 7.2,\n        \"max\": 9.1,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          9.1,\n          7.2\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Placed\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"No\",\n          \"Yes\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\",\n  \"variable_name\": \"student_df\"\n}"

# Core libraries
import numpy as np
import pandas as pd

```

```

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Preprocessing (used later for scaling & encoding)
from sklearn.preprocessing import (
    StandardScaler, MinMaxScaler, RobustScaler, Normalizer,
    LabelEncoder, OneHotEncoder, OrdinalEncoder
)

# Display / plotting settings
%matplotlib inline
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (7, 4)
pd.set_option("display.max_columns", None)

print("All libraries imported successfully!")

All libraries imported successfully!

# Load all datasets we will use in this notebook
titanic = sns.load_dataset("titanic")
iris = sns.load_dataset("iris")
tips = sns.load_dataset("tips")
penguins = sns.load_dataset("penguins")

print("Titanic :", titanic.shape)
print("Iris :", iris.shape)
print("Tips :", tips.shape)
print("Penguins:", penguins.shape)

Titanic : (891, 15)
Iris : (150, 5)
Tips : (244, 7)
Penguins: (344, 7)

# California Housing – a larger, purely numerical dataset (fetched
from scikit-learn)
from sklearn.datasets import fetch_california_housing

try:
    housing_bunch = fetch_california_housing(as_frame=True)
    housing = housing_bunch.frame
except Exception as e:
    # Fallback: if there is no internet access, build a similar
    synthetic dataset
    print("Could not download California Housing, generating a similar
    synthetic dataset instead.")
    from sklearn.datasets import make_regression
    X, y = make_regression(n_samples=2000, n_features=8, noise=15,

```

```

random_state=42)
cols = ["MedInc", "HouseAge", "AveRooms", "AveBedrms",
        "Population", "AveOccup", "Latitude", "Longitude"]
housing = pd.DataFrame(X, columns=cols)
housing["MedHouseVal"] = y

```

```

print("California Housing:", housing.shape)
housing.head()

```

California Housing: (20640, 9)

```

{"summary": "{\n  \"name\": \"housing\",\n  \"rows\": 20640,\n  \"fields\": [\n    {\n      \"column\": \"MedInc\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1.8998217179452732,\n        \"min\": 0.4999,\n        \"max\": 15.0001,\n        \"num_unique_values\": 12928,\n        \"samples\": [\n          5.0286,\n          2.0433,\n          6.1228\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"HouseAge\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 12.585557612111637,\n        \"min\": 1.0,\n        \"max\": 52.0,\n        \"num_unique_values\": 52,\n        \"samples\": [\n          35.0,\n          25.0,\n          7.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"AveRooms\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 2.4741731394243205,\n        \"min\": 0.8461538461538461,\n        \"max\": 141.9090909090909,\n        \"num_unique_values\": 19392,\n        \"samples\": [\n          6.111269614835948,\n          5.912820512820513,\n          5.7924528301886795\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"AveBedrms\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.47391085679546435,\n        \"min\": 0.3333333333333333,\n        \"max\": 34.06666666666667,\n        \"num_unique_values\": 14233,\n        \"samples\": [\n          0.9906542056074766,\n          1.112099644128114,\n          1.0398230088495575\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Population\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1132.4621217653375,\n        \"min\": 3.0,\n        \"max\": 35682.0,\n        \"num_unique_values\": 3888,\n        \"samples\": [\n          4169.0,\n          636.0,\n          3367.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"AveOccup\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 10.386049562213591,\n        \"min\": 0.6923076923076923,\n        \"max\": 1243.3333333333333,\n        \"num_unique_values\": 18841,\n        \"samples\": [\n          2.6939799331103678,\n          3.559375,\n          3.297082228116711\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}

```

```

\"description\": \"\"\n    }\n  },\n  {\n    \"column\":  

\"Latitude\", \n    \"properties\": {\n      \"dtype\":  

\"number\", \n      \"std\": 2.1359523974571117, \n      \"min\":  

32.54, \n      \"max\": 41.95, \n      \"num_unique_values\": 862, \n  

\"samples\": [\n        33.7, \n        34.41, \n        38.24 \n  

], \n      \"semantic_type\": \"\", \n      \"description\": \"\" \n  

}\n  }, \n  {\n    \"column\": \"Longitude\", \n  

\"properties\": {\n      \"dtype\": \"number\", \n      \"std\":  

2.003531723502581, \n      \"min\": -124.35, \n      \"max\": -  

114.31, \n      \"num_unique_values\": 844, \n      \"samples\": [\n  

-118.63, \n      -119.86, \n      -121.26 \n      ], \n  

\"semantic_type\": \"\", \n      \"description\": \"\" \n    } \n  

  }, \n  {\n    \"column\": \"MedHouseVal\", \n  

\"properties\": {\n      \"dtype\": \"number\", \n      \"std\":  

1.1539561587441483, \n      \"min\": 0.14999, \n      \"max\":  

5.00001, \n      \"num_unique_values\": 3842, \n      \"samples\":  

[\n        1.943, \n        3.79, \n        2.301 \n      ], \n  

\"semantic_type\": \"\", \n      \"description\": \"\" \n    } \n  

  } \n ] \n }\", \"type\": \"dataframe\", \"variable_name\": \"housing\"}

```

```

import seaborn as sns
sns.get_dataset_names()

```

```

['anagrams',
 'anscombe',
 'attention',
 'brain_networks',
 'car_crashes',
 'diamonds',
 'dots',
 'dowjones',
 'exercise',
 'flights',
 'fmri',
 'geyser',
 'glue',
 'healthexp',
 'iris',
 'mpg',
 'penguins',
 'planets',
 'seaice',
 'taxis',
 'tips',
 'titanic']

```

```

import plotly.express as px
dir(px.data)

```

```
['_all_',  
 '__builtins__',  
 '__cached__',  
 '__doc__',  
 '__file__',  
 '__loader__',  
 '__name__',  
 '__package__',  
 '__path__',  
 '__spec__',  
 'carshare',  
 'election',  
 'election_geojson',  
 'experiment',  
 'gapminder',  
 'iris',  
 'medals_long',  
 'medals_wide',  
 'stocks',  
 'tips',  
 'wind']
```

```
import seaborn as sns  
dir(sns)
```

```
['FacetGrid',  
 'JointGrid',  
 'PairGrid',  
 '__builtins__',  
 '__cached__',  
 '__doc__',  
 '__file__',  
 '__loader__',  
 '__name__',  
 '__package__',  
 '__path__',  
 '__spec__',  
 '__version__',  
 '_base',  
 '_compat',  
 '_core',  
 '_docstrings',  
 '_orig_rc_params',  
 '_statistics',  
 '_stats',  
 '_algorithms',  
 '_axes_style',  
 '_axisgrid',  
 '_barplot',  
 '_blend_palette',
```

```
'boxenplot',  
'boxplot',  
'categorical',  
'catplot',  
'choose_colorbrewer_palette',  
'choose_cubehelix_palette',  
'choose_dark_palette',  
'choose_diverging_palette',  
'choose_light_palette',  
'clustermap',  
'cm',  
'color_palette',  
'colors',  
'countplot',  
'crayon_palette',  
'crayons',  
'cubehelix_palette',  
'dark_palette',  
'desaturate',  
'despine',  
'displot',  
'distplot',  
'distributions',  
'diverging_palette',  
'dogplot',  
'ecdfplot',  
'external',  
'get_data_home',  
'get_dataset_names',  
'heatmap',  
'histplot',  
'hls_palette',  
'husl_palette',  
'jointplot',  
'kdeplot',  
'light_palette',  
'lineplot',  
'lmplot',  
'load_dataset',  
'matrix',  
'miscplot',  
'move_legend',  
'mpl',  
'mpl_palette',  
'pairplot',  
'palettes',  
'palplot',  
'plotting_context',  
'pointplot',
```

```

'rcmod',
'regplot',
'regression',
'relational',
'relplot',
'reset_defaults',
'reset_orig',
'residplot',
'rugplot',
'saturate',
'scatterplot',
'set',
'set_color_codes',
'set_context',
'set_hls_values',
'set_palette',
'set_style',
'set_theme',
'stripplot',
'swarmplot',
'utils',
'violinplot',
'widgets',
'xkcd_palette',
'xkcd_rgb']

```

```
titanic.head()    # first 5 rows – a quick peek at the data
```

```

{"summary": "{\n  \"name\": \"titanic\",\n  \"rows\": 891,\n  \"fields\": [\n    {\n      \"column\": \"survived\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n        \"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          1,\n          0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"pclass\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 1,\n        \"max\": 3,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          3,\n          1\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"sex\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"female\",\n          \"male\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"age\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 14.526497332334044,\n        \"min\": 0.42,\n        \"max\": 80.0,\n        \"num_unique_values\": 88,\n        \"samples\": [\n          0.75,\n          22.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"\n  ]\n}"}

```

```

\"sibsp\",\\n      \"properties\": {\\n          \"dtype\": \"number\",\\n
\"std\": 1,\\n          \"min\": 0,\\n          \"max\": 8,\\n
\"num_unique_values\": 7,\\n          \"samples\": [\\n              1,\\n
0\\n          ],\\n          \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n  },\\n  {\\n      \"column\":
\"parch\",\\n      \"properties\": {\\n          \"dtype\": \"number\",\\n
\"std\": 0,\\n          \"min\": 0,\\n          \"max\": 6,\\n
\"num_unique_values\": 7,\\n          \"samples\": [\\n              0,\\n
1\\n          ],\\n          \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n  },\\n  {\\n      \"column\":
\"fare\",\\n      \"properties\": {\\n          \"dtype\": \"number\",\\n
\"std\": 49.693428597180905,\\n          \"min\": 0.0,\\n          \"max\":
512.3292,\\n          \"num_unique_values\": 248,\\n          \"samples\":
[\\n              11.2417,\\n              51.8625\\n          ],\\n
\"semantic_type\": \"\",\\n          \"description\": \"\"\\n      }\\n
n  },\\n  {\\n      \"column\": \"embarked\",\\n      \"properties\":
{\\n          \"dtype\": \"category\",\\n          \"num_unique_values\":
3,\\n          \"samples\": [\\n              \"S\",\\n              \"C\"\\n
],\\n          \"semantic_type\": \"\",\\n          \"description\": \"\"\\n
}\\n  },\\n  {\\n      \"column\": \"class\",\\n      \"properties\":
{\\n          \"dtype\": \"category\",\\n          \"num_unique_values\":
3,\\n          \"samples\": [\\n              \"Third\",\\n              \"First\"\\n
n          ],\\n          \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n  },\\n  {\\n      \"column\":
\"who\",\\n      \"properties\": {\\n          \"dtype\": \"category\",\\n
\"num_unique_values\": 3,\\n          \"samples\": [\\n              \"man\",\\n
n              \"woman\"\\n          ],\\n          \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n  },\\n  {\\n      \"column\":
\"adult_male\",\\n      \"properties\": {\\n          \"dtype\":
\"boolean\",\\n          \"num_unique_values\": 2,\\n          \"samples\":
[\\n              false,\\n              true\\n          ],\\n
\"semantic_type\": \"\",\\n          \"description\": \"\"\\n      }\\n
n  },\\n  {\\n      \"column\": \"deck\",\\n      \"properties\": {\\n
\"dtype\": \"category\",\\n          \"num_unique_values\": 7,\\n
\"samples\": [\\n              \"C\",\\n              \"E\"\\n          ],\\n
\"semantic_type\": \"\",\\n          \"description\": \"\"\\n      }\\n
n  },\\n  {\\n      \"column\": \"embark_town\",\\n      \"properties\":
{\\n          \"dtype\": \"category\",\\n          \"num_unique_values\": 3,\\n
\"samples\": [\\n              \"Southampton\",\\n              \"Cherbourg\"\\n
],\\n          \"semantic_type\": \"\",\\n          \"description\": \"\"\\n
n      },\\n      {\\n          \"column\": \"alive\",\\n          \"properties\": {\\n
n              \"dtype\": \"category\",\\n              \"num_unique_values\": 2,\\n
\"samples\": [\\n                  \"yes\",\\n                  \"no\"\\n          ],\\n
\"semantic_type\": \"\",\\n          \"description\": \"\"\\n      }\\n
n      },\\n      {\\n          \"column\": \"alone\",\\n          \"properties\": {\\n
n              \"dtype\": \"boolean\",\\n              \"num_unique_values\": 2,\\n
\"samples\": [\\n                  true,\\n                  false\\n          ],\\n

```

```
\ "semantic_type\": \ "\",\n      \ "description\": \ "\n      }\n    }\n  ]\n}", "type": "dataframe", "variable_name": "titanic"}
```

```
titanic.tail() # last 5 rows – useful to check if data ends
correctly (no junk rows)
```

```
{ "summary": "{\n  \ "name\": \ "titanic",\n  \ "rows\": 5,\n  \ "fields\":\n  [\n    {\n      \ "column\": \ "survived",\n      \ "properties\": {\n        \ "dtype\": \ "number",\n        \ "std\": 0,\n        \ "min\": 0,\n        \ "max\": 1,\n        \ "num_unique_values\": 2,\n        \ "samples\":\n        [\n          1,\n          0\n        ],\n        \ "semantic_type\":\n        \ "\",\n        \ "description\": \ "\n      },\n      {\n        \ "column\": \ "pclass",\n        \ "properties\": {\n          \ "dtype\":\n          \ "number",\n          \ "std\": 1,\n          \ "min\": 1,\n          \ "max\": 3,\n          \ "num_unique_values\": 3,\n          \ "samples\":\n          [\n            2,\n            1\n          ],\n          \ "semantic_type\":\n          \ "\",\n          \ "description\": \ "\n        },\n        {\n          \ "column\": \ "sex",\n          \ "properties\": {\n            \ "dtype\":\n            \ "category",\n            \ "num_unique_values\": 2,\n            \ "samples\":\n            [\n              \ "female",\n              \ "male"\n            ],\n            \ "semantic_type\": \ "\",\n            \ "description\": \ "\n          },\n          {\n            \ "column\": \ "age",\n            \ "properties\": {\n              \ "dtype\": \ "number",\n              \ "std\": 5.354126134736337,\n              \ "min\": 19.0,\n              \ "max\": 32.0,\n              \ "num_unique_values\":\n              4,\n              \ "samples\": [\n                19.0,\n                32.0\n              ],\n              \ "semantic_type\": \ "\",\n              \ "description\": \ "\n            },\n            {\n              \ "column\":\n              \ "sibsp",\n              \ "properties\": {\n                \ "dtype\": \ "number",\n                \ "std\": 0,\n                \ "min\": 0,\n                \ "max\": 1,\n                \ "num_unique_values\": 2,\n                \ "samples\": [\n                  1,\n                  0\n                ],\n                \ "semantic_type\": \ "\",\n                \ "description\": \ "\n              },\n              {\n                \ "column\":\n                \ "parch",\n                \ "properties\": {\n                  \ "dtype\": \ "number",\n                  \ "std\": 0,\n                  \ "min\": 0,\n                  \ "max\": 2,\n                  \ "num_unique_values\": 2,\n                  \ "samples\": [\n                    2,\n                    0\n                  ],\n                  \ "semantic_type\": \ "\",\n                  \ "description\": \ "\n                },\n                {\n                  \ "column\":\n                  \ "fare",\n                  \ "properties\": {\n                    \ "dtype\": \ "number",\n                    \ "std\": 10.09253436952285,\n                    \ "min\": 7.75,\n                    \ "max\":\n                    30.0,\n                    \ "num_unique_values\": 4,\n                    \ "samples\": [\n                      30.0,\n                      7.75\n                    ],\n                    \ "semantic_type\": \ "\",\n                    \ "description\": \ "\n                  },\n                  {\n                    \ "column\":\n                    \ "embarked",\n                    \ "properties\": {\n                      \ "dtype\":\n                      \ "string",\n                      \ "num_unique_values\": 3,\n                      \ "samples\":\n                      [\n                        \ "S",\n                        \ "C"\n                      ],\n                      \ "semantic_type\": \ "\",\n                      \ "description\": \ "\n                    },\n                    {\n                      \ "column\": \ "class",\n                      \ "properties\": {\n                        \ "dtype\": \ "category",\n                        \ "num_unique_values\": 3,\n                        \ "samples\": [\n                          \ "Second",\n                          \ "First"\n                        ],\n                        \ "semantic_type\": \ "\",\n
```

```

{"description": {"column": "adult_male", "properties": {"dtype": "boolean", "num_unique_values": 2, "samples": [false, true]}, "semantic_type": "boolean"}, {"column": "deck", "properties": {"dtype": "category", "num_unique_values": 2, "samples": ["C", "B"]}, "semantic_type": "category"}, {"column": "embark_town", "properties": {"dtype": "string", "num_unique_values": 3, "samples": ["Southampton", "Cherbourg"]}, "semantic_type": "string"}, {"column": "alive", "properties": {"dtype": "category", "num_unique_values": 2, "samples": ["yes", "no"]}, "semantic_type": "category"}, {"column": "alone", "properties": {"dtype": "boolean", "num_unique_values": 2, "samples": [false, true]}, "semantic_type": "boolean"}], "type": "dataframe"}

```

`titanic.sample(5)` # 5 random rows – helps you see rows `head()`/`tail()` might miss

```

{"summary": {"name": "titanic", "rows": 5, "fields": [{"column": "survived", "properties": {"dtype": "number", "std": 0, "min": 0, "max": 1, "num_unique_values": 2, "samples": [0, 1]}, "semantic_type": "boolean"}, {"column": "pclass", "properties": {"dtype": "number", "std": 0, "min": 1, "max": 3, "num_unique_values": 2, "samples": [3, 1]}, "semantic_type": "category"}, {"column": "sex", "properties": {"dtype": "category", "num_unique_values": 2, "samples": ["male", "female"]}, "semantic_type": "category"}, {"column": "age", "properties": {"dtype": "number", "std": 22.64950330581225, "min": 18.0, "max": 63.0, "num_unique_values": 3, "samples": [18.0, 63.0]}]}]}

```

```

n      ],\n          \"semantic_type\": \"\",\n\n\"description\": \"\"\n      },\n      {\n          \"column\":\n\n\"sibsp\",\n          \"properties\": {\n              \"dtype\": \"number\",\n\n\"std\": 0,\n              \"min\": 1,\n              \"max\": 2,\n\n\"num_unique_values\": 2,\n              \"samples\": [\n                  1,\n                  2\n              ],\n          \"semantic_type\": \"\",\n\n\"description\": \"\"\n      },\n      {\n          \"column\":\n\n\"parch\",\n          \"properties\": {\n              \"dtype\": \"number\",\n\n\"std\": 0,\n              \"min\": 0,\n              \"max\": 2,\n\n\"num_unique_values\": 3,\n              \"samples\": [\n                  2,\n                  0\n              ],\n          \"semantic_type\": \"\",\n\n\"description\": \"\"\n      },\n      {\n          \"column\":\n\n\"fare\",\n          \"properties\": {\n              \"dtype\": \"number\",\n\n\"std\": 91.18376868841845,\n              \"min\": 22.3583,\n              \"max\": 262.375,\n              \"num_unique_values\": 5,\n              \"samples\": [\n                  89.1042,\n                  78.85\n              ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n      },\n      {\n          \"column\": \"embarked\",\n          \"properties\": {\n              \"dtype\": \"category\",\n              \"num_unique_values\":\n\n2,\n              \"samples\": [\n                  \"S\",\n                  \"C\"\n              ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n      },\n      {\n          \"column\": \"class\",\n          \"properties\": {\n              \"dtype\": \"category\",\n              \"num_unique_values\":\n\n2,\n              \"samples\": [\n                  \"Third\",\n                  \"First\"\n              ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n      },\n      {\n          \"column\":\n\n\"who\",\n          \"properties\": {\n              \"dtype\": \"category\",\n\n\"num_unique_values\": 2,\n              \"samples\": [\n                  \"man\",\n                  \"woman\"\n              ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n      },\n      {\n          \"column\":\n\n\"adult_male\",\n          \"properties\": {\n              \"dtype\":\n\n\"boolean\",\n              \"num_unique_values\": 2,\n              \"samples\":\n\n[\n                  true,\n                  false\n              ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n      },\n      {\n          \"column\": \"deck\",\n          \"properties\": {\n              \"dtype\": \"category\",\n              \"num_unique_values\": 4,\n              \"samples\": [\n                  \"C\",\n                  \"D\"\n              ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n      },\n      {\n          \"column\": \"embark_town\",\n          \"properties\": {\n              \"dtype\": \"category\",\n\n\"num_unique_values\": 2,\n              \"samples\": [\n                  \"Southampton\",\n                  \"Cherbourg\"\n              ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n      },\n      {\n          \"column\": \"alive\",\n          \"properties\": {\n              \"dtype\": \"category\",\n              \"num_unique_values\": 2,\n              \"samples\": [\n                  \"no\",\n                  \"yes\"\n              ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n      },\n      {\n          \"column\": \"alone\",\n          \"properties\": {\n              \"dtype\": \"boolean\",\n              \"num_unique_values\": 1,\n
```

```
\n        false\n    ],\n    \"semantic_type\": \"\",\n    \"description\": \"\"\n}\n    ],\n    \"type\": \"dataframe\"}
```

```
print("Shape (rows, columns):", titanic.shape)
print("Size (total values) :", titanic.size)
print("Dimensions (ndim)    :", titanic.ndim)
```

```
Shape (rows, columns): (891, 15)
Size (total values)   : 13365
Dimensions (ndim)     : 2
```

```
print("Columns:\n", list(titanic.columns))
print("\nIndex:", titanic.index)
```

```
Columns:
['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
'alive', 'alone']
```

```
Index: RangeIndex(start=0, stop=891, step=1)
```

```
# Let's classify Titanic's columns ourselves
titanic.dtypes
```

```
survived      int64
pclass        int64
sex           object
age           float64
sibsp         int64
parch         int64
fare          float64
embarked      object
class         category
who           object
adult_male    bool
deck          category
embark_town   object
alive         object
alone         bool
dtype: object
```

```
titanic.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null   int64
1   pclass      891 non-null   int64
```

```

2  sex          891 non-null  object
3  age          714 non-null  float64
4  sibsp       891 non-null  int64
5  parch       891 non-null  int64
6  fare        891 non-null  float64
7  embarked    889 non-null  object
8  class       891 non-null  category
9  who         891 non-null  object
10 adult_male  891 non-null  bool
11 deck        203 non-null  category
12 embark_town 889 non-null  object
13 alive       891 non-null  object
14 alone       891 non-null  bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB

```

```

titanic.describe()  # summary statistics for NUMERICAL columns only

{"summary": "{\n  \"name\": \"titanic\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"survived\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 314.8713661874558,\n        \"min\": 0.0,\n        \"max\": 891.0,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          0.3838383838383838,\n          1.0,\n          0.4865924542648585\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"pclass\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 314.2523437079693,\n        \"min\": 0.8360712409770513,\n        \"max\": 891.0,\n        \"num_unique_values\": 6,\n        \"samples\": [\n          891.0,\n          2.308641975308642,\n          3.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"age\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 242.9056731818781,\n        \"min\": 0.42,\n        \"max\": 714.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          29.69911764705882,\n          28.0,\n          714.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"sibsp\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 314.4908277465442,\n        \"min\": 0.0,\n        \"max\": 891.0,\n        \"num_unique_values\": 6,\n        \"samples\": [\n          891.0,\n          0.5230078563411896,\n          8.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"parch\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 314.65971717879,\n        \"min\": 0.0,\n        \"max\": 891.0,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          0.38159371492704824,\n          6.0,\n          0.8060572211299559\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"fare\",\n      \"properties\": {\n

```

```

\"dtype\": \"number\", \n          \"std\": 330.6256632228577, \n
\"min\": 0.0, \n          \"max\": 891.0, \n          \"num_unique_values\":
8, \n          \"samples\": [ \n          32.204207968574636, \n
14.4542, \n          891.0 \n          ], \n          \"semantic_type\":
\"\", \n          \"description\": \"\" \n          } \n          ] \n
n} \", \"type\": \"dataframe\"}

```

```

titanic.describe(include="all") # summary for ALL columns (numeric +
categorical)

```

```

{"summary": "{ \n  \"name\": \"titanic\", \n  \"rows\": 11, \n
  \"fields\": [ \n    { \n      \"column\": \"survived\", \n
      \"properties\": { \n        \"dtype\": \"number\", \n        \"std\":
314.8713661874558, \n        \"min\": 0.0, \n        \"max\": 891.0, \n
        \"num_unique_values\": 5, \n        \"samples\": [ \n
0.3838383838383838, \n        1.0, \n        0.4865924542648585 \n
], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n
} \n      }, \n      { \n        \"column\": \"pclass\", \n        \"properties\":
{ \n          \"dtype\": \"number\", \n          \"std\":
314.2523437079693, \n          \"min\": 0.8360712409770513, \n
          \"max\": 891.0, \n          \"num_unique_values\": 6, \n
          \"samples\": [ \n            891.0, \n            2.308641975308642, \n
3.0 \n          ], \n          \"semantic_type\": \"\", \n
          \"description\": \"\" \n          } \n        }, \n        { \n          \"column\":
\"sex\", \n          \"properties\": { \n            \"dtype\": \"category\", \n
            \"num_unique_values\": 4, \n            \"samples\": [ \n
2, \n            \"577\", \n            \"891\" \n          ], \n            \"semantic_type\":
\"\", \n            \"description\": \"\" \n          } \n        }, \n        { \n          \"column\":
\"age\", \n          \"properties\": { \n            \"dtype\":
\"number\", \n            \"std\": 242.9056731818781, \n            \"min\":
0.42, \n            \"max\": 714.0, \n            \"num_unique_values\": 8, \n
            \"samples\": [ \n              29.69911764705882, \n              28.0, \n
714.0 \n            ], \n            \"semantic_type\": \"\", \n
            \"description\": \"\" \n          } \n        }, \n        { \n          \"column\":
\"sibsp\", \n          \"properties\": { \n            \"dtype\": \"number\", \n
            \"std\": 314.4908277465442, \n            \"min\": 0.0, \n            \"max\":
891.0, \n            \"num_unique_values\": 6, \n            \"samples\": [ \n
891.0, \n            0.5230078563411896, \n            8.0 \n          ], \n            \"semantic_type\":
\"\", \n            \"description\": \"\" \n          } \n        }, \n        { \n          \"column\":
\"parch\", \n          \"properties\": { \n            \"dtype\": \"number\", \n
            \"std\": 314.65971717879, \n            \"min\": 0.0, \n            \"max\":
891.0, \n            \"num_unique_values\": 5, \n            \"samples\": [ \n
0.38159371492704824, \n            6.0, \n            0.8060572211299559 \n          ], \n            \"semantic_type\":
\"\", \n            \"description\": \"\" \n          } \n        }, \n        { \n          \"column\":
\"fare\", \n          \"properties\": { \n            \"dtype\": \"number\", \n
            \"std\": 330.6256632228577, \n            \"min\": 0.0, \n            \"max\":
891.0, \n            \"num_unique_values\": 8, \n            \"samples\": [ \n
32.204207968574636, \n            14.4542, \n            891.0 \n          ], \n            \"semantic_type\":

```

```

{"description": "\n    },\n    {\n    \"column\": \"embarked\",\n    \"properties\": {\n    \"dtype\":\n    \"category\",\n    \"num_unique_values\": 4,\n    \"samples\":\n    [\n    3,\n    \"644\",\n    \"889\",\n    ],\n    \"semantic_type\": \"\",\n    \"description\": \"\n    }\n    },\n    {\n    \"column\": \"class\",\n    \"properties\": {\n    \"dtype\": \"category\",\n    \"num_unique_values\": 4,\n    \"samples\": [\n    3,\n    \"491\",\n    \"891\",\n    ],\n    \"semantic_type\": \"\",\n    \"description\": \"\n    }\n    },\n    {\n    \"column\": \"who\",\n    \"properties\": {\n    \"dtype\": \"category\",\n    \"num_unique_values\": 4,\n    \"samples\": [\n    3,\n    \"537\",\n    \"891\",\n    ],\n    \"semantic_type\": \"\",\n    \"description\": \"\n    }\n    },\n    {\n    \"column\": \"adult_male\",\n    \"properties\": {\n    \"dtype\": \"category\",\n    \"num_unique_values\": 4,\n    \"samples\": [\n    2,\n    \"537\",\n    \"891\",\n    ],\n    \"semantic_type\":\n    \"\",\n    \"description\": \"\n    }\n    },\n    {\n    \"column\": \"deck\",\n    \"properties\": {\n    \"dtype\":\n    \"category\",\n    \"num_unique_values\": 4,\n    \"samples\":\n    [\n    7,\n    \"59\",\n    \"203\",\n    ],\n    \"semantic_type\": \"\",\n    \"description\": \"\n    }\n    },\n    {\n    \"column\": \"embark_town\",\n    \"properties\": {\n    \"dtype\": \"category\",\n    \"num_unique_values\": 4,\n    \"samples\": [\n    3,\n    \"644\",\n    \"889\",\n    ],\n    \"semantic_type\":\n    \"\",\n    \"description\": \"\n    }\n    },\n    {\n    \"column\": \"alone\",\n    \"properties\": {\n    \"dtype\": \"category\",\n    \"num_unique_values\": 4,\n    \"samples\": [\n    2,\n    \"537\",\n    \"891\",\n    ],\n    \"semantic_type\": \"\",\n    \"description\": \"\n    }\n    }\n    ],\n    \"type\": \"dataframe\"}

```

```

age = titanic["age"].dropna() # drop missing values just for this calculation

```

```

print("Mean          :", round(age.mean(), 2))
print("Median         :", round(age.median(), 2))
print("Mode           :", age.mode()[0])
print("Variance        :", round(age.var(), 2))
print("Standard Deviation :", round(age.std(), 2))
print("Q1 (25th percentile):", age.quantile(0.25))
print("Q3 (75th percentile):", age.quantile(0.75))
print("90th Percentile  :", age.quantile(0.90))
print("Range           :", age.max() - age.min())

```

```

print("IQR (Q3 - Q1)          :", age.quantile(0.75) -
age.quantile(0.25))

Mean          : 29.7
Median        : 28.0
Mode          : 24.0
Variance      : 211.02
Standard Deviation : 14.53
Q1 (25th percentile): 20.125
Q3 (75th percentile): 38.0
90th Percentile : 50.0
Range         : 79.58
IQR (Q3 - Q1) : 17.875

missing_count = titanic.isnull().sum()
missing_percent = (titanic.isnull().sum() / len(titanic) *
100).round(2)

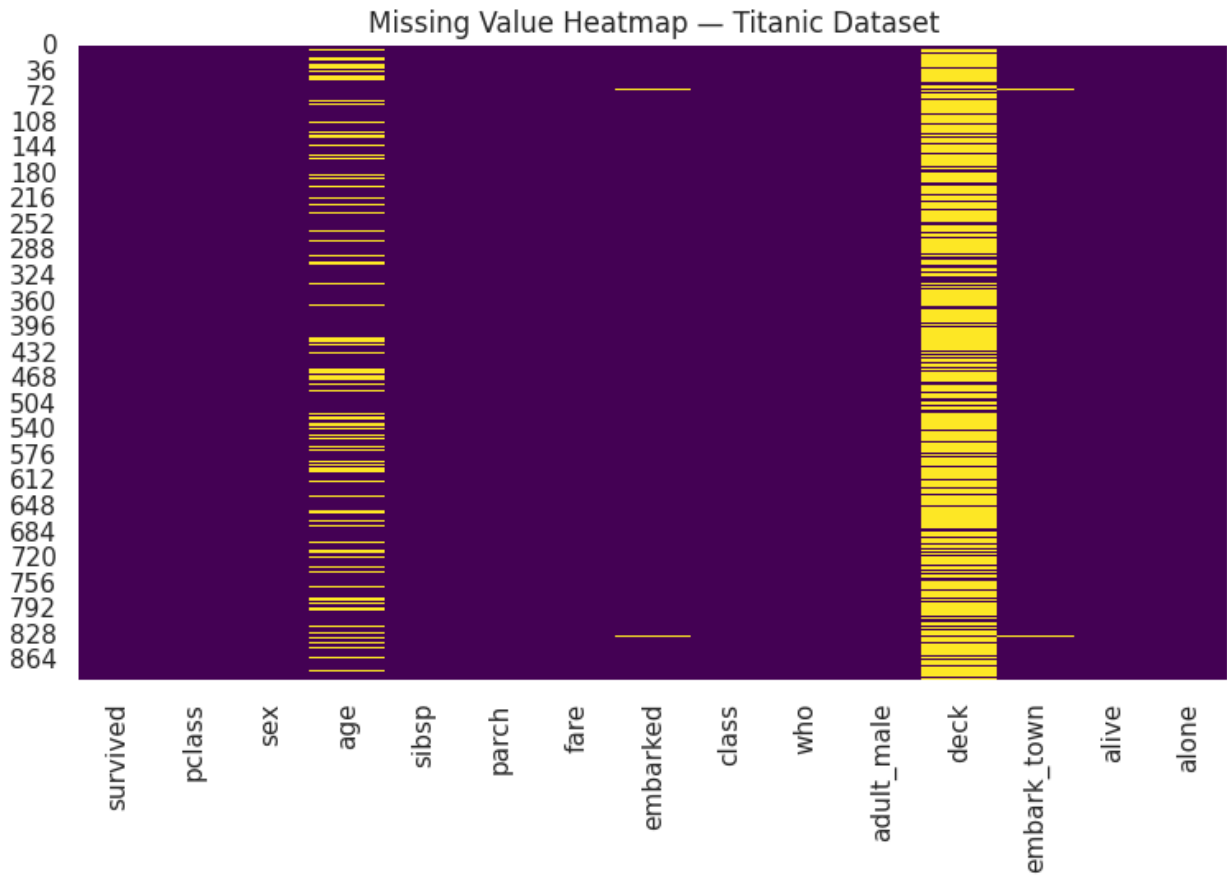
missing_summary = pd.DataFrame({
    "Missing Count": missing_count,
    "Missing %": missing_percent
}).sort_values("Missing %", ascending=False)

missing_summary[missing_summary["Missing Count"] > 0]

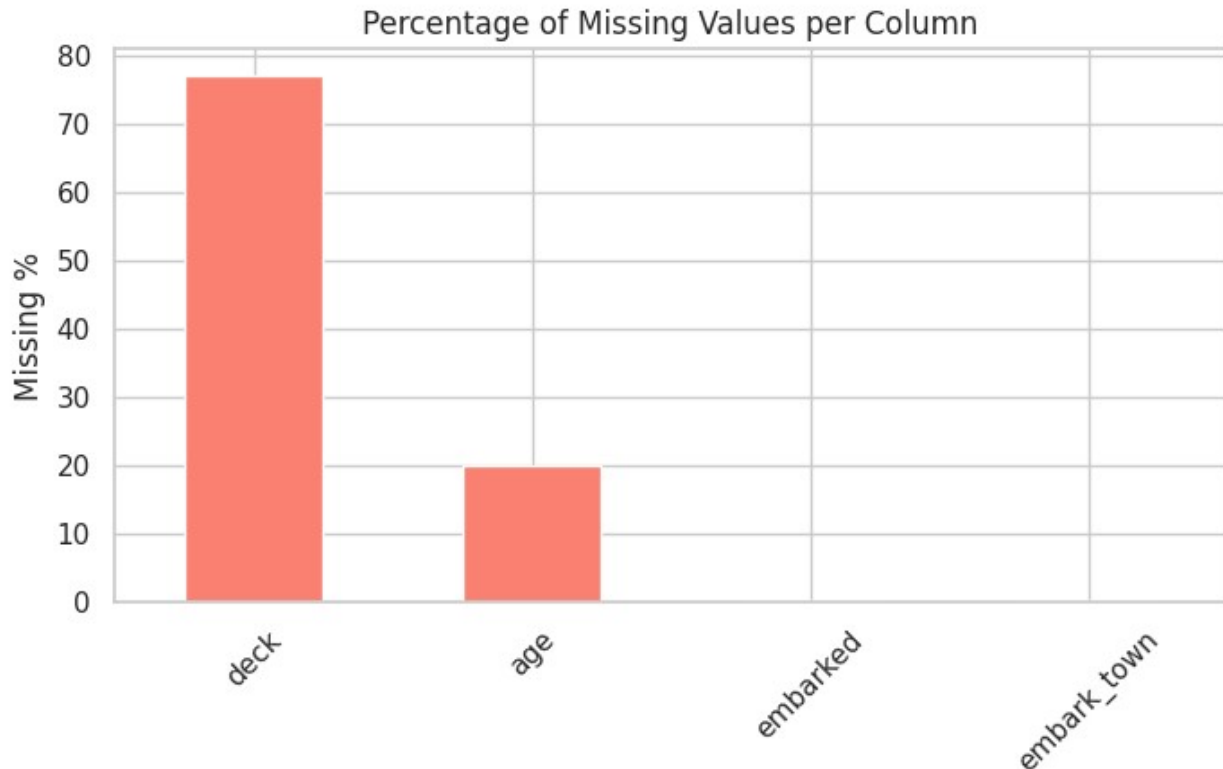
{"summary": "{\n  \"name\":\n  \"missing_summary[missing_summary[\\\"Missing Count\\\"] > 0]\",\n  \"rows\": 4,\n  \"fields\": [\n    {\n      \"column\": \"Missing\nCount\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 324,\n        \"min\": 2,\n        \"max\": 688,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          688,\n          177,\n          2\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\":\n      \"Missing %\",\n      \"properties\": {\n        \"dtype\":\n        \"number\",\n        \"std\": 36.42259772448967,\n        \"min\":\n        0.22,\n        \"max\": 77.22,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          77.22,\n          19.87,\n          0.22\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\"}

# Visualization 1: Heatmap of missing values
plt.figure(figsize=(9, 5))
sns.heatmap(titanic.isnull(), cbar=False, cmap="viridis")
plt.title("Missing Value Heatmap – Titanic Dataset")
plt.show()

```



```
# Visualization 2: Bar chart of missing value percentage
plt.figure(figsize=(8, 4))
missing_summary[missing_summary["Missing Count"] > 0]["Missing
%"].plot(kind="bar", color="salmon")
plt.ylabel("Missing %")
plt.title("Percentage of Missing Values per Column")
plt.xticks(rotation=45)
plt.show()
```



```
df_demo = titanic.copy()  # always work on a COPY, never the
                           original!

# 1. Drop rows with any missing value (use sparingly – can lose a lot
of data)
df_drop = df_demo.dropna()
print("After dropna():", df_drop.shape)

# 2. Fill numerical column with MEAN
df_demo["age_mean_filled"] =
df_demo["age"].fillna(df_demo["age"].mean())

# 3. Fill numerical column with MEDIAN (better when data is skewed /
has outliers)
df_demo["age_median_filled"] =
df_demo["age"].fillna(df_demo["age"].median())

# 4. Fill categorical column with MODE (most frequent value)
df_demo["embarked_mode_filled"] =
df_demo["embarked"].fillna(df_demo["embarked"].mode()[0])

# 5. Forward Fill – carry the previous valid value forward
df_demo["age_ffill"] = df_demo["age"].ffill()

# 6. Backward Fill – carry the next valid value backward
df_demo["age_bfill"] = df_demo["age"].bfill()
```

```
# 7. Interpolation – estimate missing values using surrounding data
df_demo["age_interpolated"] = df_demo["age"].interpolate()
```

```
# 8. Fill with a CONSTANT value
```

```
df_demo["deck_constant_filled"] =
df_demo["deck"].astype("object").fillna("Unknown")
```

```
df_demo[["age", "age_mean_filled", "age_median_filled",
         "age_ffill", "age_bfill", "age_interpolated"]].head(8)
```

```
After dropna(): (182, 15)
```

```
{
  "summary": {
    "name": "age_ffill",
    "rows": 8,
    "fields": [
      {
        "column": "age",
        "properties": {
          "dtype": "number",
          "std": 16.090517763725376,
          "min": 2.0,
          "max": 54.0,
          "num_unique_values": 6,
          "samples": [22.0, 38.0, 2.0]
        },
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "age_mean_filled",
        "properties": {
          "dtype": "number",
          "std": 14.898368343513292,
          "min": 2.0,
          "max": 54.0,
          "num_unique_values": 7,
          "samples": [22.0, 38.0, 54.0]
        },
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "age_median_filled",
        "properties": {
          "dtype": "number",
          "std": 14.91882798920306,
          "min": 2.0,
          "max": 54.0,
          "num_unique_values": 7,
          "samples": [22.0, 38.0, 54.0]
        },
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "age_ffill",
        "properties": {
          "dtype": "number",
          "std": 14.989877536905649,
          "min": 2.0,
          "max": 54.0,
          "num_unique_values": 6,
          "samples": [22.0, 38.0, 2.0]
        },
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "age_bfill",
        "properties": {
          "dtype": "number",
          "std": 17.09427640218461,
          "min": 2.0,
          "max": 54.0,
          "num_unique_values": 6,
          "samples": [22.0, 38.0, 2.0]
        },
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "age_interpolated",
        "properties": {
          "dtype": "number",
          "std": 15.721771755844095,
          "min": 2.0,
          "max": 54.0,
          "num_unique_values": 7,
          "samples": [22.0, 38.0, 54.0]
        },
        "semantic_type": "",
        "description": ""
      }
    ],
    "type": "dataframe"
  }
}
```

```

print("Number of duplicate rows in Titanic:",
titanic.duplicated().sum())

# Let's create an artificial example to see duplicates in action
demo = titanic.head(5).copy()
demo = pd.concat([demo, demo.iloc[[0, 2]]], ignore_index=True) #
manually duplicate 2 rows
print("\nDemo dataset shape (with duplicates):", demo.shape)
print("Duplicate rows found:", demo.duplicated().sum())

demo[demo.duplicated(keep=False)] # show all copies of duplicated
rows

```

Number of duplicate rows in Titanic: 107

Demo dataset shape (with duplicates): (7, 15)

Duplicate rows found: 2

```

{"summary":{"\n  \"name\": \"demo[demo\", \"rows\": 4,\n  \"fields\": [\n    {\n      \"column\": \"survived\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0,\n        \"min\": 0, \n        \"max\": 1, \n        \"num_unique_values\": 2, \n        \"samples\": [\n          1, \n          0\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }, \n      {\n        \"column\": \"pclass\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 0, \n          \"min\": 3, \n          \"max\": 3, \n          \"num_unique_values\": 1, \n          \"samples\": [\n            3\n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\"\n        }, \n        {\n          \"column\": \"sex\", \n          \"properties\": {\n            \"dtype\": \"string\", \n            \"num_unique_values\": 2, \n            \"samples\": [\n              \"female\"\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n          }, \n          {\n            \"column\": \"age\", \n            \"properties\": {\n              \"dtype\": \"number\", \n              \"std\": 2.309401076758503, \n              \"min\": 22.0, \n              \"max\": 26.0, \n              \"num_unique_values\": 2, \n              \"samples\": [\n                26.0\n              ], \n              \"semantic_type\": \"\", \n              \"description\": \"\"\n            }, \n            {\n              \"column\": \"sibsp\", \n              \"properties\": {\n                \"dtype\": \"number\", \n                \"std\": 0, \n                \"min\": 0, \n                \"max\": 1, \n                \"num_unique_values\": 2, \n                \"samples\": [\n                  0\n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\"\n              }, \n              {\n                \"column\": \"parch\", \n                \"properties\": {\n                  \"dtype\": \"number\", \n                  \"std\": 0, \n                  \"min\": 0, \n                  \"max\": 0, \n                  \"num_unique_values\": 1, \n                  \"samples\": [\n                    0\n                  ], \n                  \"semantic_type\": \"\", \n                  \"description\": \"\"\n                }, \n                {\n                  \"column\": \"fare\", \n                  \"properties\": {\n                    \"dtype\": \"number\", \n                    \"std\": 0.38971143170299727, \n                    \"min\": 7.25, \n

```

```

{"max": 7.925, "num_unique_values": 2, "samples": [{"max": 7.925}], "semantic_type": {"description": ""}, {"column": "embarked", "properties": {"dtype": "category", "num_unique_values": 1, "samples": [{"S"}], "semantic_type": {"description": ""}, {"column": "class", "properties": {"dtype": "category", "num_unique_values": 1, "samples": [{"Third"}], "semantic_type": {"description": ""}, {"column": "who", "properties": {"dtype": "string", "num_unique_values": 2, "samples": [{"woman"}], "semantic_type": {"description": ""}, {"column": "adult_male", "properties": {"dtype": "boolean", "num_unique_values": 2, "samples": [{"false"}], "semantic_type": {"description": ""}, {"column": "deck", "properties": {"dtype": "category", "num_unique_values": 0, "samples": []}, "semantic_type": {"description": ""}, {"column": "embark_town", "properties": {"dtype": "category", "num_unique_values": 1, "samples": []}, "semantic_type": {"description": ""}, {"column": "alive", "properties": {"dtype": "string", "num_unique_values": 2, "samples": []}, "semantic_type": {"description": ""}, {"column": "alone", "properties": {"dtype": "boolean", "num_unique_values": 2, "samples": []}, "semantic_type": {"description": ""}}], "type": "dataframe"}

```

```

demo_cleaned = demo.drop_duplicates()
print("Shape before:", demo.shape, " | Shape after drop_duplicates():", demo_cleaned.shape)

```

Shape before: (7, 15) | Shape after drop\_duplicates(): (5, 15)

```

print("Unique classes in 'pclass':", titanic["pclass"].unique())
print("Number of unique classes :", titanic["pclass"].nunique())

```

```

print("\nValue counts for 'embarked':")
print(titanic["embarked"].value_counts())

```

```

print("\nValue counts (as %) for 'sex':")
print(titanic["sex"].value_counts(normalize=True).round(3) * 100)

```

```
Unique classes in 'pclass': [3 1 2]
Number of unique classes : 3
```

```
Value counts for 'embarked':
```

```
embarked
```

```
S    644
```

```
C    168
```

```
Q     77
```

```
Name: count, dtype: int64
```

```
Value counts (as %) for 'sex':
```

```
sex
```

```
male    64.8
```

```
female  35.2
```

```
Name: proportion, dtype: float64
```

```
numeric_titanic = titanic.select_dtypes(include=np.number)
```

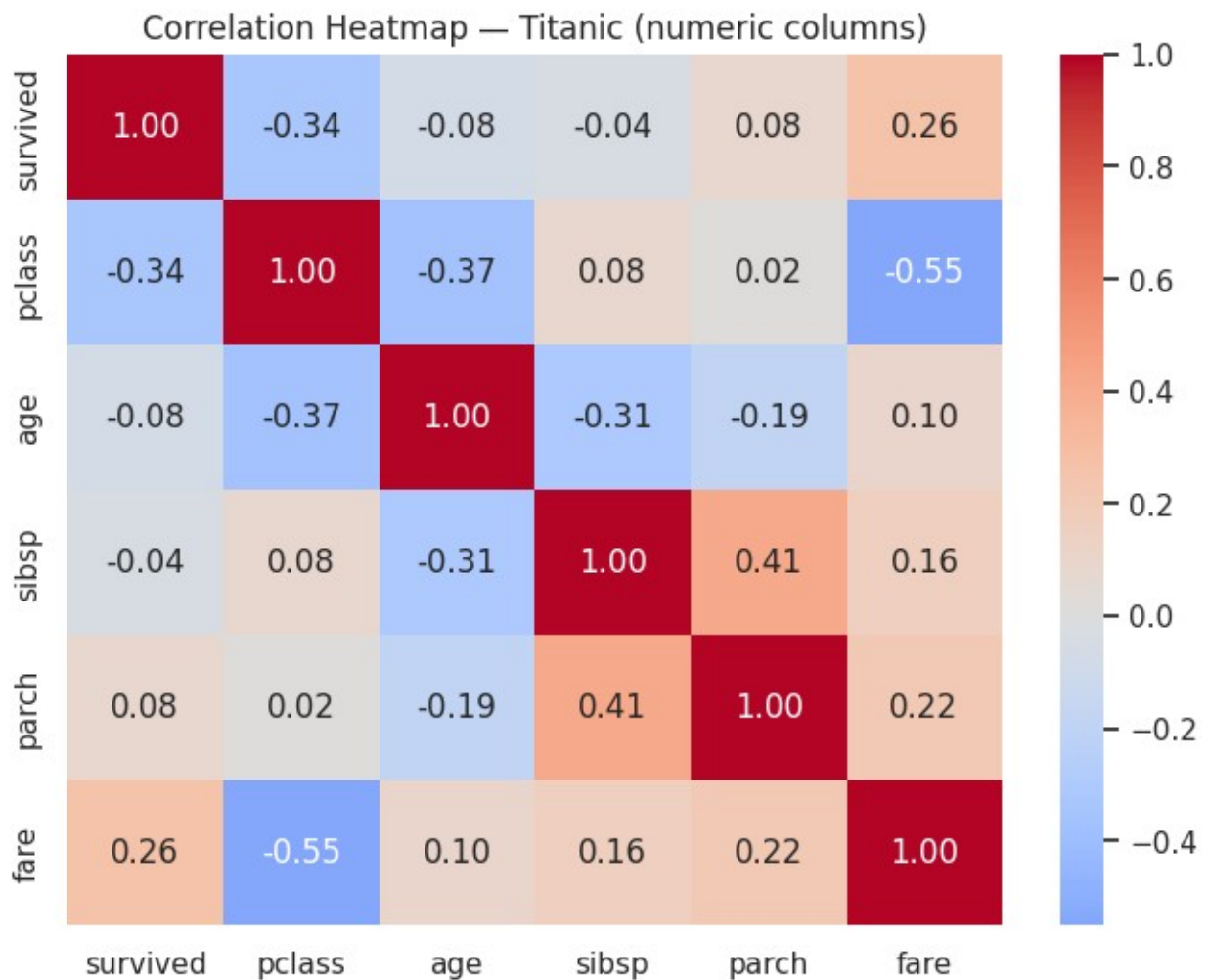
```
corr_matrix = numeric_titanic.corr()
```

```
corr_matrix
```

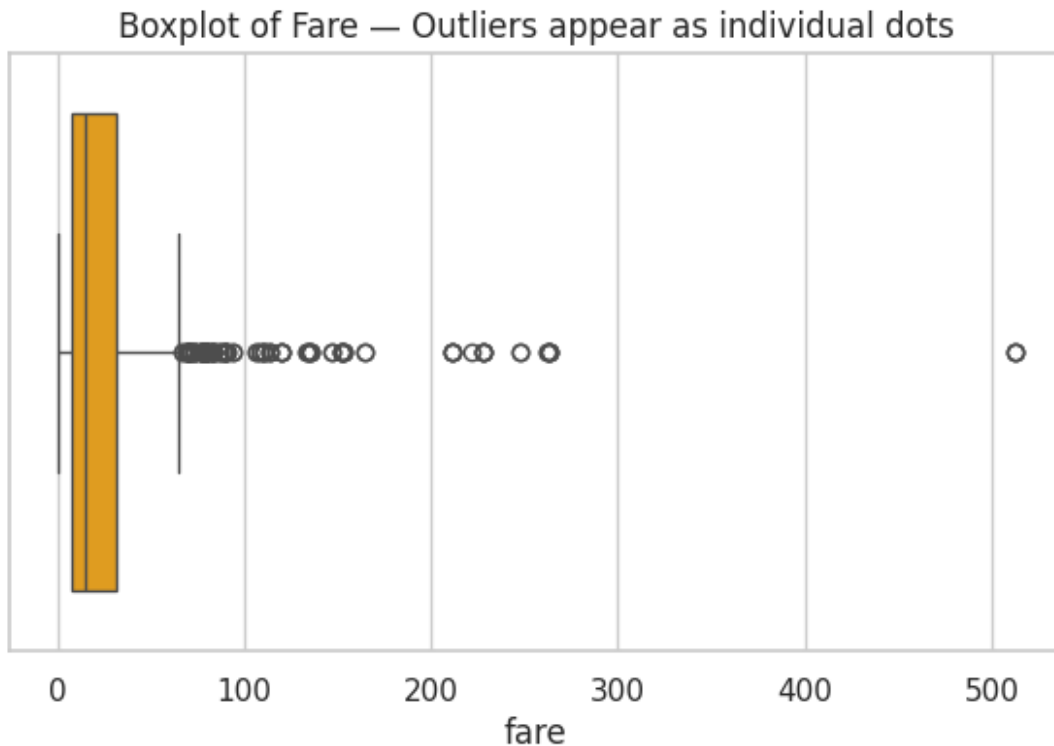
```
{"summary": "{\n  \"name\": \"corr_matrix\",\n  \"rows\": 6,\n  \"fields\": [\n    {\n      \"column\": \"survived\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.46114042297686436,\n        \"min\": -0.33848103596101514,\n        \"max\": 1.0,\n        \"num_unique_values\": 6,\n        \"samples\": [\n          1.0,\n          -0.33848103596101514,\n          0.2573065223849626\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"pclass\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.557862030518841,\n        \"min\": -0.5494996199439076,\n        \"max\": 1.0,\n        \"num_unique_values\": 6,\n        \"samples\": [\n          -0.33848103596101514,\n          1.0,\n          0.5494996199439076\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"age\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.5056556916612195,\n        \"min\": -0.36922601531551735,\n        \"max\": 1.0,\n        \"num_unique_values\": 6,\n        \"samples\": [\n          -0.07722109457217756,\n          -0.36922601531551735,\n          0.09606669176903912\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"sibsp\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.45010635566048934,\n        \"min\": -0.30824675892365666,\n        \"max\": 1.0,\n        \"num_unique_values\": 6,\n        \"samples\": [\n          -0.035322498885735576,\n          0.08308136284568686,\n          0.159651043242161\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"parch\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.41600749623472333,\n        \"min\": -0.1891192626320352,\n        \"max\": 1.0,\n        \"num_unique_values\": 6,\n        \"samples\":
```

```
[\\n          0.08162940708348335,\\n          0.018442671310748508,\\n
0.21622494477076448\\n          ],\\n          \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n          }\\n          },\\n          {\\n          \\\"column\\\":
\\\"fare\\\",\\n          \\\"properties\\\": {\\n          \\\"dtype\\\": \\\"number\\\",\\n
\\\"std\\\": 0.49348936988307834,\\n          \\\"min\\\": -0.5494996199439076,\\n
\\\"max\\\": 1.0,\\n          \\\"num_unique_values\\\": 6,\\n          \\\"samples\\\":
[\\n          0.2573065223849626,\\n          -0.5494996199439076,\\n
1.0\\n          ],\\n          \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n          }\\n          }\\n          ]\\n
n}\\\", \"type\": \"dataframe\", \"variable_name\": \"corr_matrix\"}

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, fmt=\".2f\", cmap=\"coolwarm\",
center=0)
plt.title(\"Correlation Heatmap – Titanic (numeric columns)\")
plt.show()
```



```
plt.figure(figsize=(7, 4))
sns.boxplot(x=titanic["fare"], color="orange")
plt.title("Boxplot of Fare — Outliers appear as individual dots")
plt.show()
```



```
Q1 = titanic["fare"].quantile(0.25)
Q3 = titanic["fare"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = titanic[(titanic["fare"] < lower_bound) | (titanic["fare"]
> upper_bound)]
print(f"Lower bound: {lower_bound:.2f}    Upper bound:
{upper_bound:.2f}")
print("Number of outliers in 'fare':", outliers.shape[0])

Lower bound: -26.72    Upper bound: 65.63
Number of outliers in 'fare': 116

from scipy import stats

fare_no_na = titanic["fare"].dropna()
z_scores = np.abs(stats.zscore(fare_no_na))
```

```

z_outliers = fare_no_na[z_scores > 3]      # points more than 3 std-
devs from the mean
print("Number of outliers using Z-score (|z| > 3):",
z_outliers.shape[0])

```

Number of outliers using Z-score ( $|z| > 3$ ): 20

```

titanic_no_outliers = titanic[(titanic["fare"] >= lower_bound) &
(titanic["fare"] <= upper_bound)]
print("Original shape :", titanic.shape)
print("After removing fare outliers:", titanic_no_outliers.shape)

```

Original shape : (891, 15)  
After removing fare outliers: (775, 15)

```

housing_num = housing.select_dtypes(include=np.number).dropna()
sample_cols = housing_num.columns[:2]    # take 2 numeric columns to
compare visually

```

```

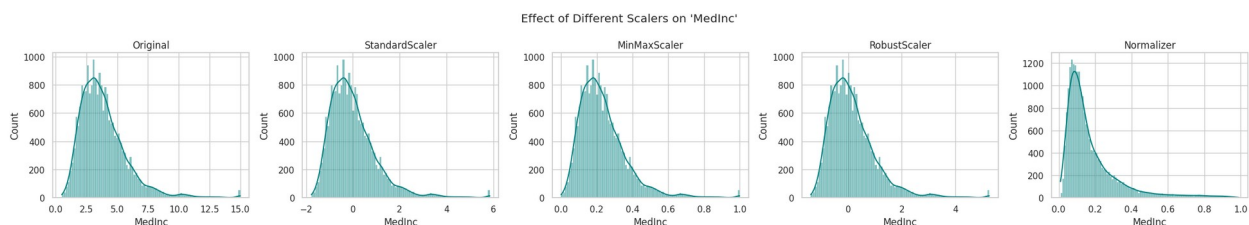
scalers = {
    "Original":      housing_num[sample_cols],
    "StandardScaler":
pd.DataFrame(StandardScaler().fit_transform(housing_num[sample_cols]),
columns=sample_cols),
    "MinMaxScaler":
pd.DataFrame(MinMaxScaler().fit_transform(housing_num[sample_cols]),
columns=sample_cols),
    "RobustScaler":
pd.DataFrame(RobustScaler().fit_transform(housing_num[sample_cols]),
columns=sample_cols),
    "Normalizer":
pd.DataFrame(Normalizer().fit_transform(housing_num[sample_cols]),
columns=sample_cols),
}

```

```

fig, axes = plt.subplots(1, 5, figsize=(22, 4))
for ax, (name, data) in zip(axes, scalers.items()):
    sns.histplot(data[sample_cols[0]], ax=ax, kde=True, color="teal")
    ax.set_title(name)
plt.suptitle(f"Effect of Different Scalers on '{sample_cols[0]}'")
plt.tight_layout()
plt.show()

```



```

demo_enc = titanic[["sex", "embarked", "pclass"]].dropna().copy()

# 1. Label Encoding – assigns each category an integer (0, 1, 2, ...)
le = LabelEncoder()
demo_enc["sex_label"] = le.fit_transform(demo_enc["sex"])

# 2. One-Hot Encoding – creates a separate 0/1 column for every
category
one_hot = pd.get_dummies(demo_enc["embarked"], prefix="embarked")

# 3. Ordinal Encoding – like Label Encoding, but you CONTROL the order
# (pclass 1 = best, 3 = worst, so we keep that natural order)
ordinal = OrdinalEncoder(categories=[[1, 2, 3]])
demo_enc["pclass_ordinal"] =
ordinal.fit_transform(demo_enc[["pclass"]])

result = pd.concat([demo_enc[["sex", "sex_label"]], one_hot,
                    demo_enc[["pclass", "pclass_ordinal"]]], axis=1)
result.head()

{"summary":{"\n  \"name\": \"result\", \n  \"rows\": 889, \n
\"fields\": [\n    {\n      \"column\": \"sex\", \n
\"properties\": {\n        \"dtype\": \"category\", \n
\"num_unique_values\": 2, \n        \"samples\": [\n
\"female\", \n        \"male\" \n        ], \n
\"semantic_type\": \"\", \n        \"description\": \"\" \n        } \n
    }, \n    {\n      \"column\": \"sex_label\", \n
\"properties\": {\n        \"dtype\": \"number\", \n        \"std\":
0, \n        \"min\": 0, \n        \"max\": 1, \n
\"num_unique_values\": 2, \n        \"samples\": [\n        0, \n
1 \n        ], \n        \"semantic_type\": \"\", \n
\"description\": \"\" \n        } \n    }, \n    {\n      \"column\":
\"embarked_C\", \n        \"properties\": {\n        \"dtype\":
\"boolean\", \n        \"num_unique_values\": 2, \n        \"samples\":
[\n        true, \n        false \n        ], \n
\"semantic_type\": \"\", \n        \"description\": \"\" \n        } \n
    }, \n    {\n      \"column\": \"embarked_Q\", \n
\"properties\": {\n        \"dtype\": \"boolean\", \n
\"num_unique_values\": 2, \n        \"samples\": [\n        true, \n
false \n        ], \n        \"semantic_type\": \"\", \n
\"description\": \"\" \n        } \n    }, \n    {\n      \"column\":
\"embarked_S\", \n        \"properties\": {\n        \"dtype\":
\"boolean\", \n        \"num_unique_values\": 2, \n        \"samples\":
[\n        false, \n        true \n        ], \n
\"semantic_type\": \"\", \n        \"description\": \"\" \n        } \n
    }, \n    {\n      \"column\": \"pclass\", \n        \"properties\":
{\n        \"dtype\": \"number\", \n        \"std\": 0, \n
\"min\": 1, \n        \"max\": 3, \n        \"num_unique_values\": 3, \n
\"samples\": [\n        3, \n        1 \n        ], \n
\"semantic_type\": \"\", \n        \"description\": \"\" \n        } \n
  ] \n}

```

```

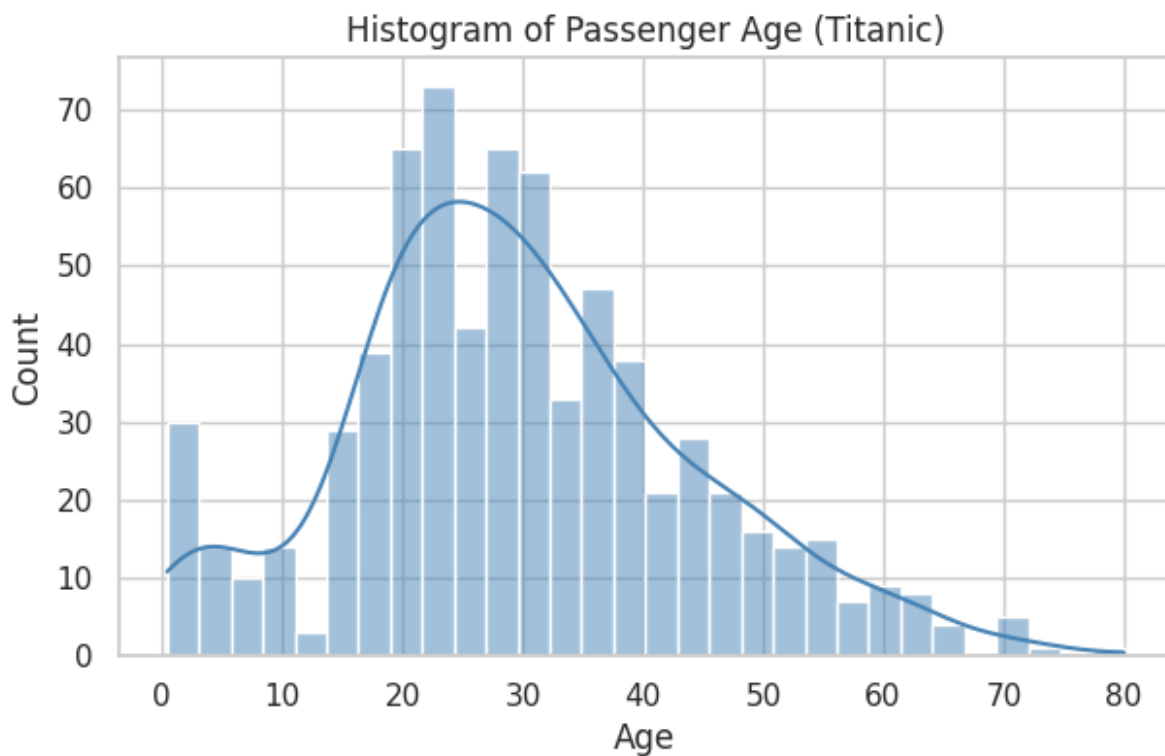
n    },\n    {\n        \"column\": \"pclass_ordinal\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 0.8346997785705772, \n            \"min\": 0.0, \n            \"max\": 2.0, \n            \"num_unique_values\": 3, \n            \"samples\": [\n                0.0\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n        }\n    }\n  ],\n  \"type\": \"dataframe\", \"variable_name\": \"result\"}

```

```

plt.figure(figsize=(7, 4))
sns.histplot(titanic[\"age\"], bins=30, kde=True, color=\"steelblue\")
plt.title(\"Histogram of Passenger Age (Titanic)\")
plt.xlabel(\"Age\")
plt.show()

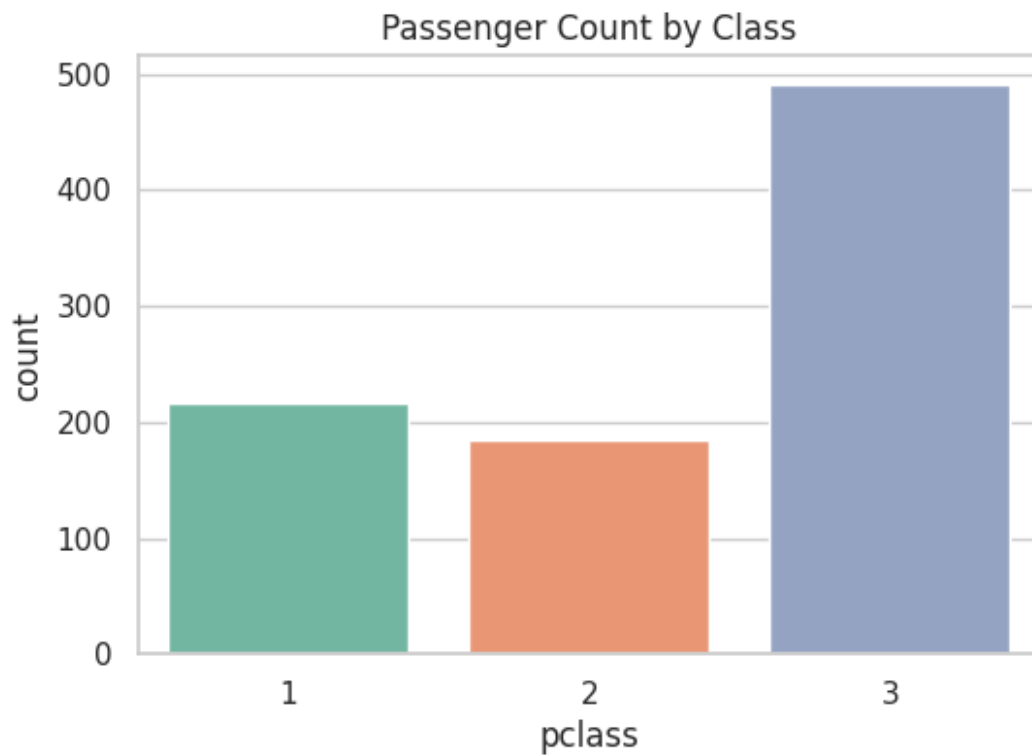
```



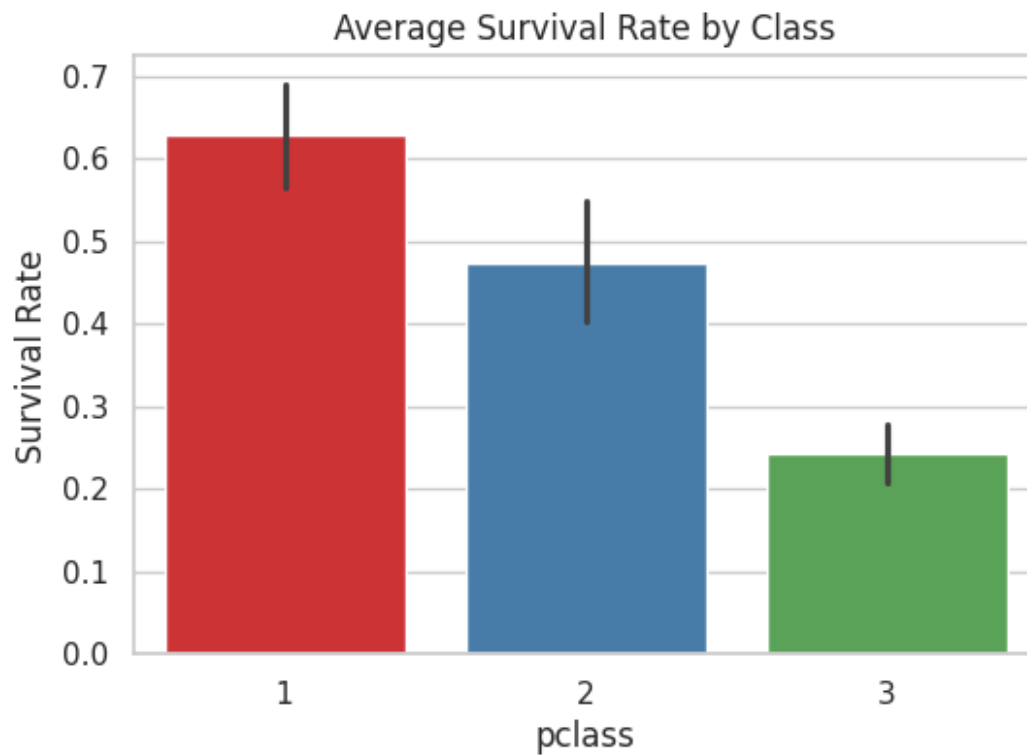
```

plt.figure(figsize=(6, 4))
sns.countplot(x="pclass", data=titanic, hue="pclass", palette="Set2",
legend=False)
plt.title("Passenger Count by Class")
plt.show()

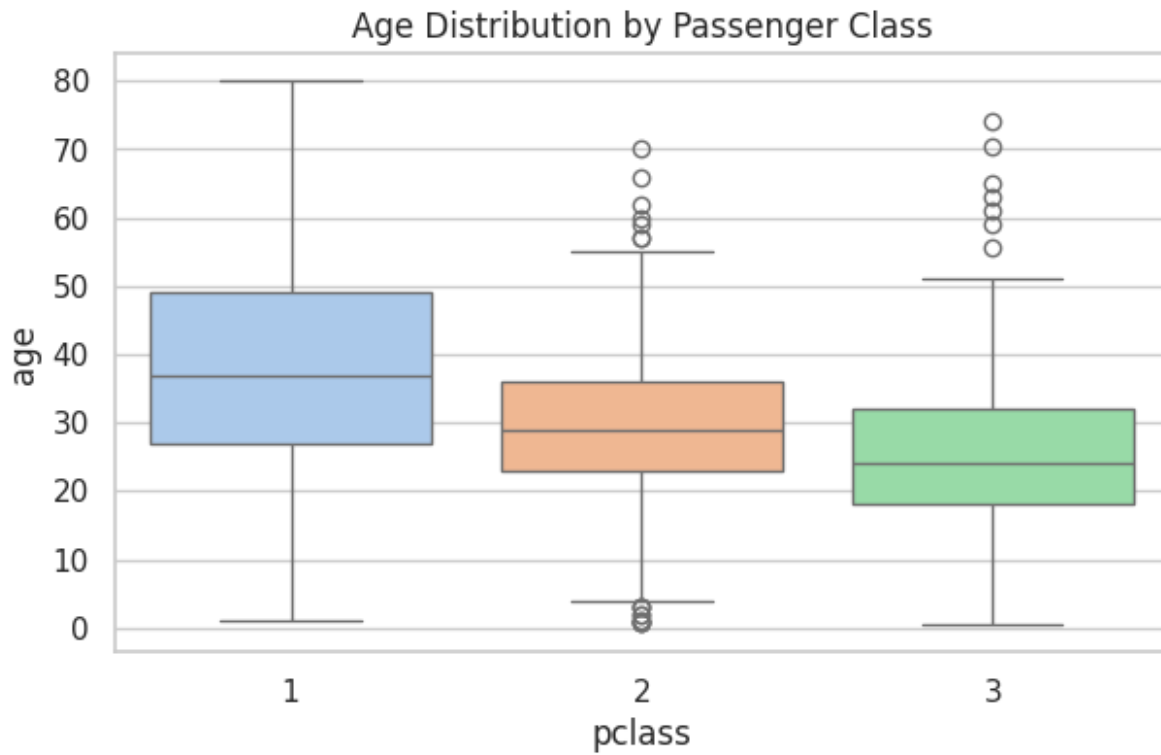
```



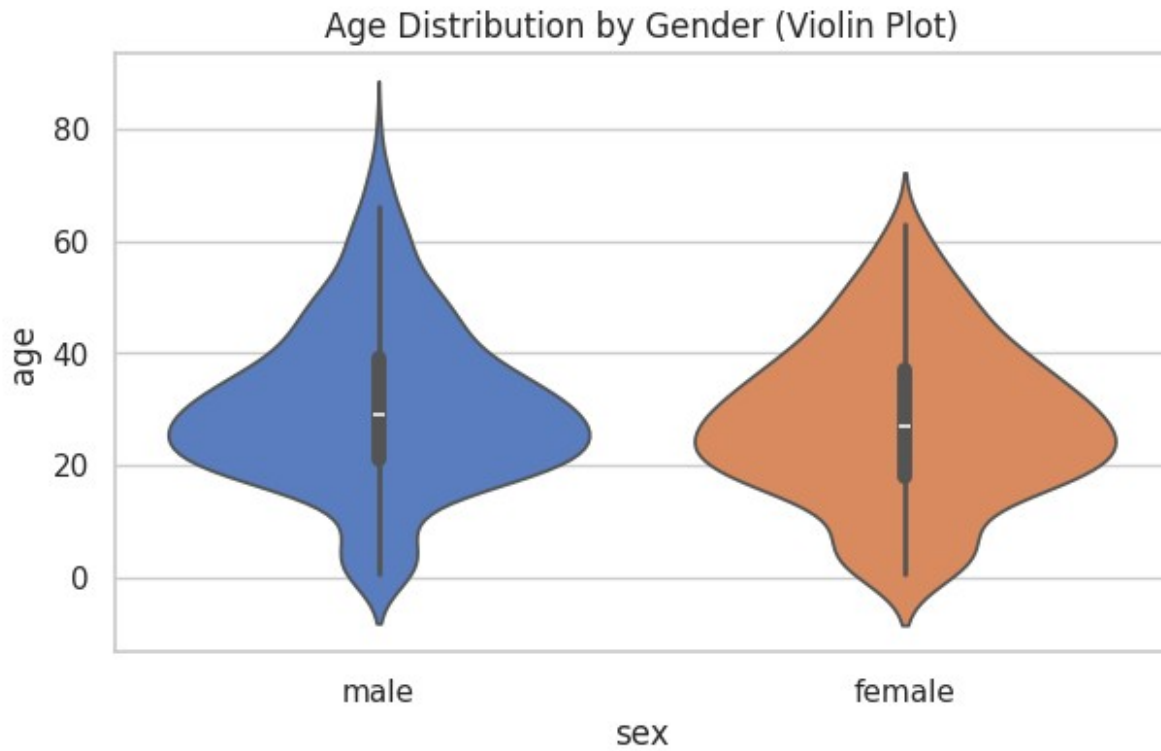
```
plt.figure(figsize=(6, 4))
sns.barplot(x="pclass", y="survived", data=titanic, hue="pclass",
palette="Set1", legend=False)
plt.title("Average Survival Rate by Class")
plt.ylabel("Survival Rate")
plt.show()
```



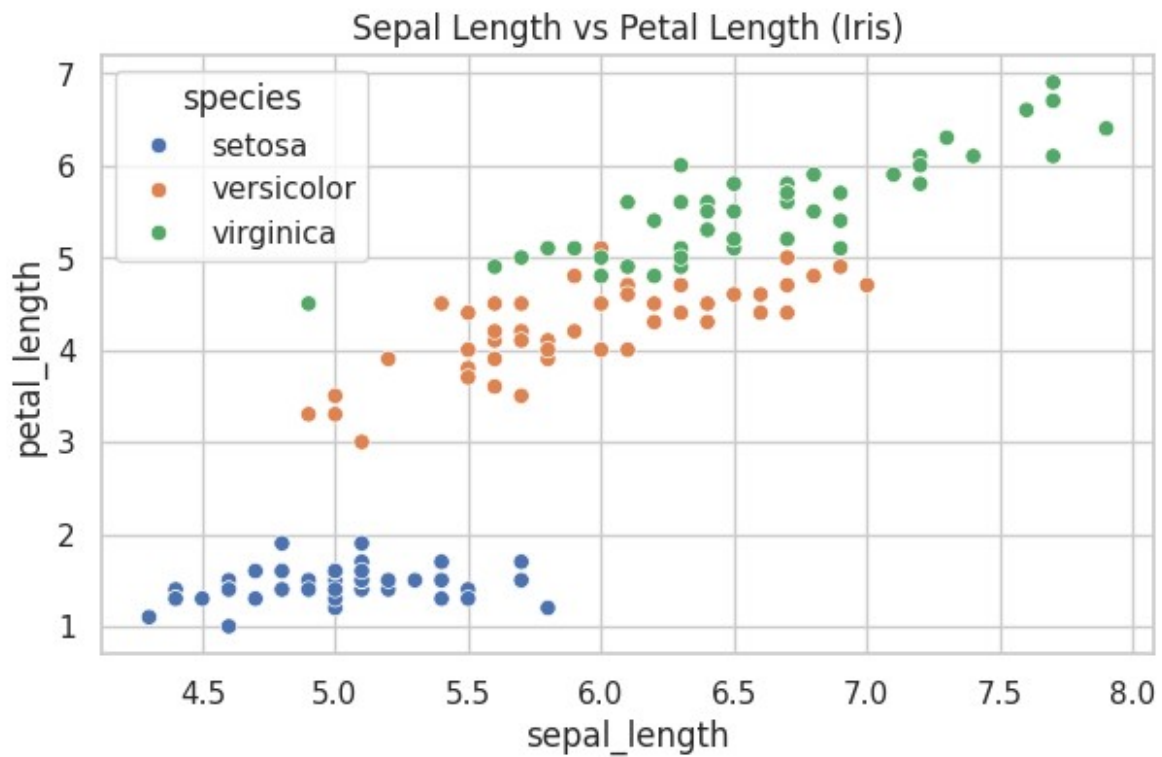
```
plt.figure(figsize=(7, 4))
sns.boxplot(x="pclass", y="age", data=titanic, hue="pclass",
palette="pastel", legend=False)
plt.title("Age Distribution by Passenger Class")
plt.show()
```



```
plt.figure(figsize=(7, 4))
sns.violinplot(x="sex", y="age", data=titanic, hue="sex",
palette="muted", legend=False)
plt.title("Age Distribution by Gender (Violin Plot)")
plt.show()
```

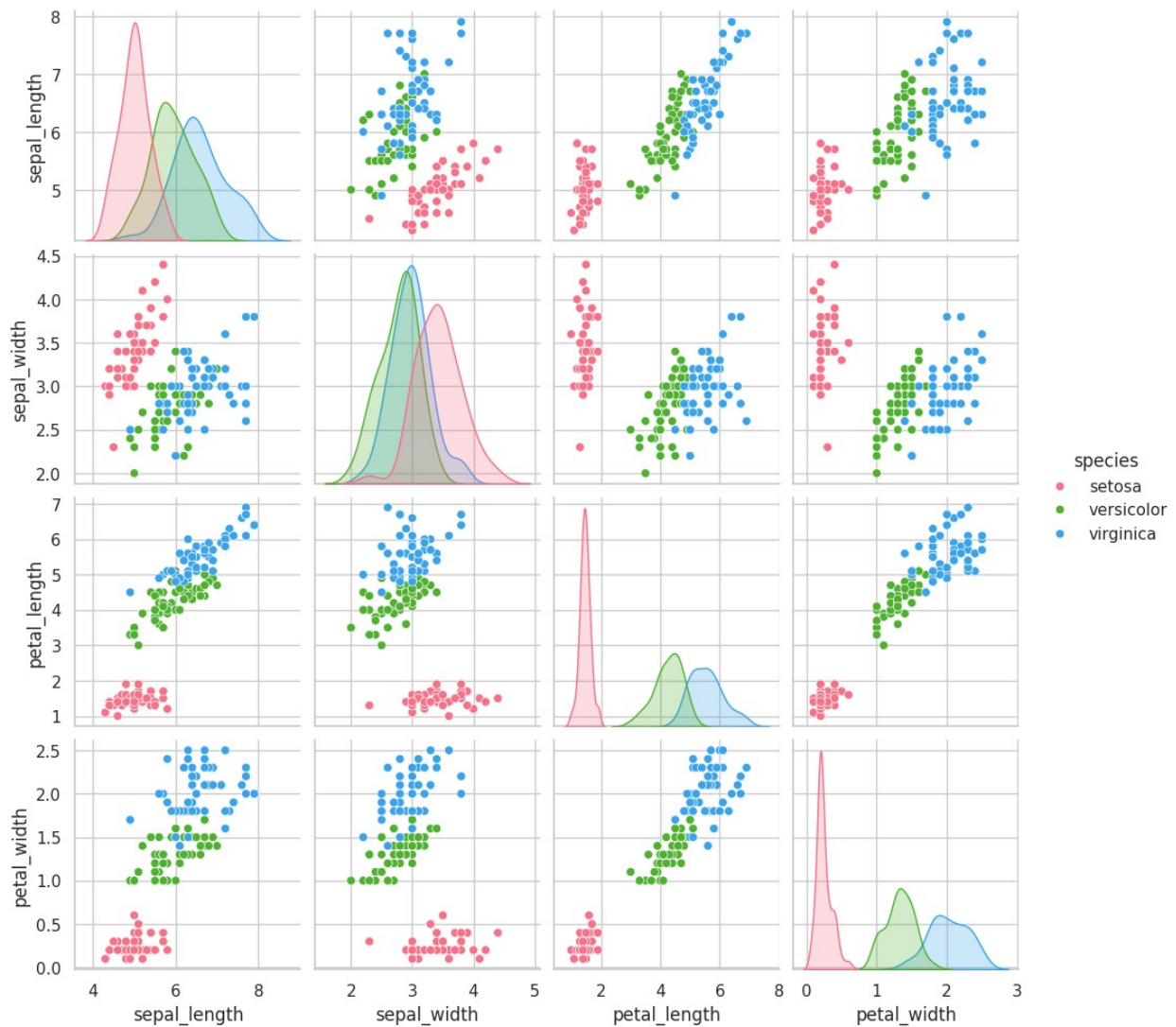


```
plt.figure(figsize=(7, 4))
sns.scatterplot(x="sepal_length", y="petal_length", hue="species",
data=iris, palette="deep")
plt.title("Sepal Length vs Petal Length (Iris)")
plt.show()
```

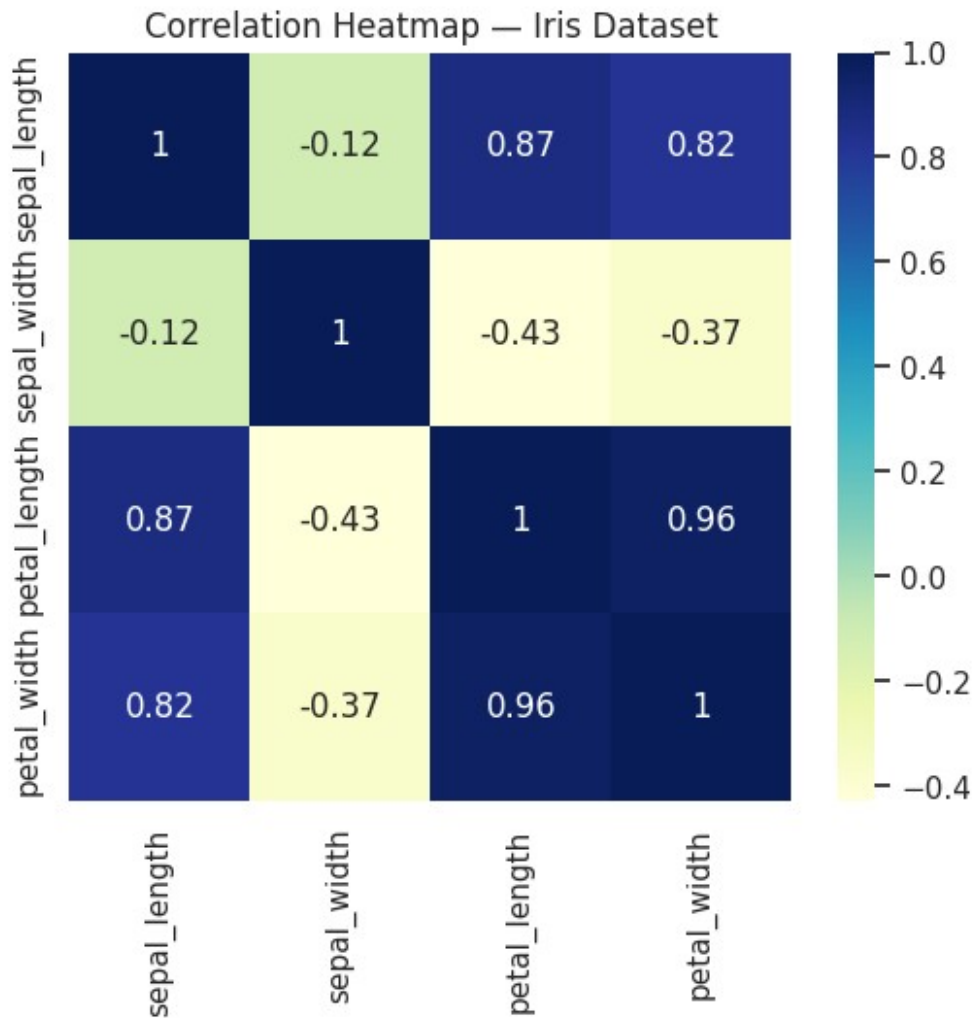


```
sns.pairplot(iris, hue="species", palette="husl", diag_kind="kde")  
plt.suptitle("Pairplot of Iris Dataset", y=1.02)  
plt.show()
```

Pairplot of Iris Dataset

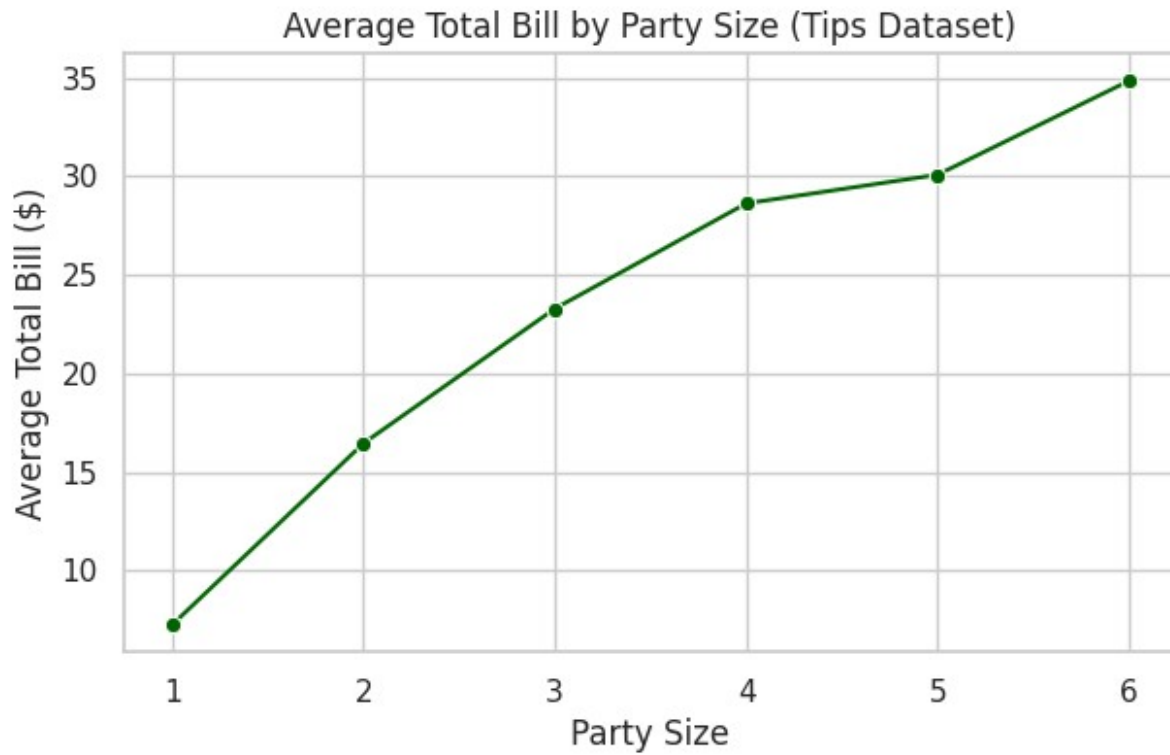


```
plt.figure(figsize=(6, 5))
sns.heatmap(iris.drop(columns="species").corr(), annot=True,
            cmap="YlGnBu")
plt.title("Correlation Heatmap – Iris Dataset")
plt.show()
```



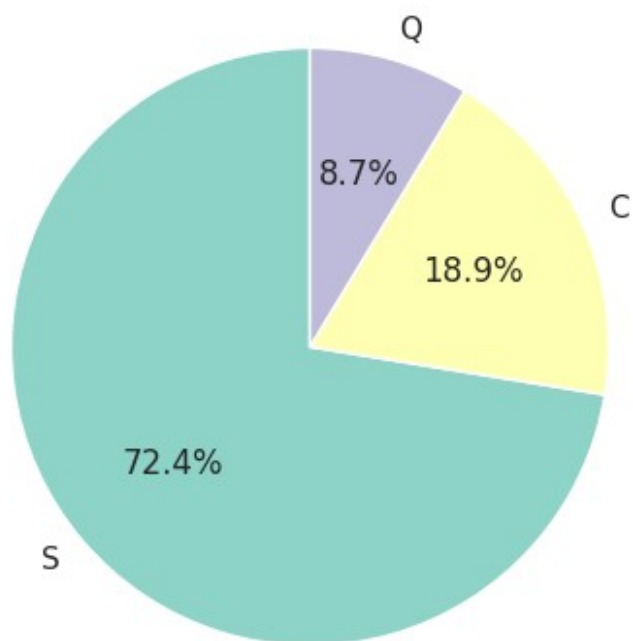
```
bill_by_size = tips.groupby("size")["total_bill"].mean().reset_index()

plt.figure(figsize=(7, 4))
sns.lineplot(x="size", y="total_bill", data=bill_by_size, marker="o",
color="darkgreen")
plt.title("Average Total Bill by Party Size (Tips Dataset)")
plt.xlabel("Party Size")
plt.ylabel("Average Total Bill ($)")
plt.show()
```

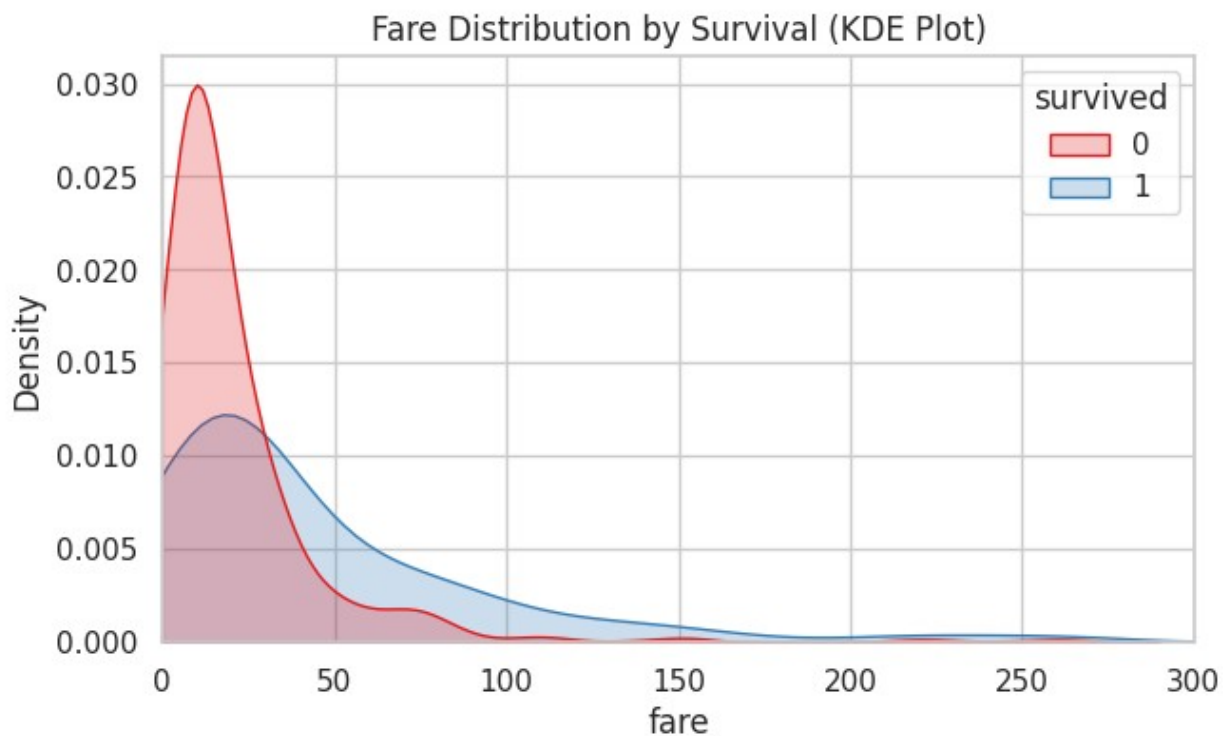


```
plt.figure(figsize=(5, 5))
titanic["embarked"].value_counts().plot(
    kind="pie", autopct="%1.1f%%", colors=sns.color_palette("Set3"),
    startangle=90)
plt.title("Passengers by Embarkation Port")
plt.ylabel("")
plt.show()
```

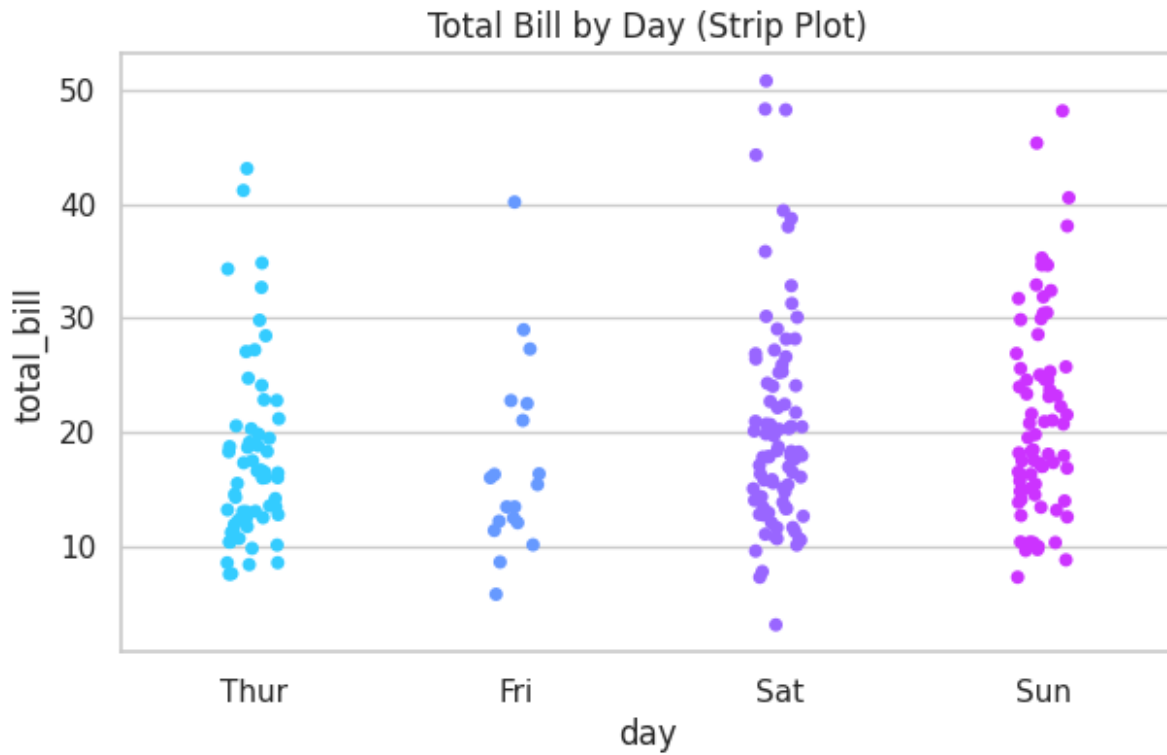
Passengers by Embarkation Port



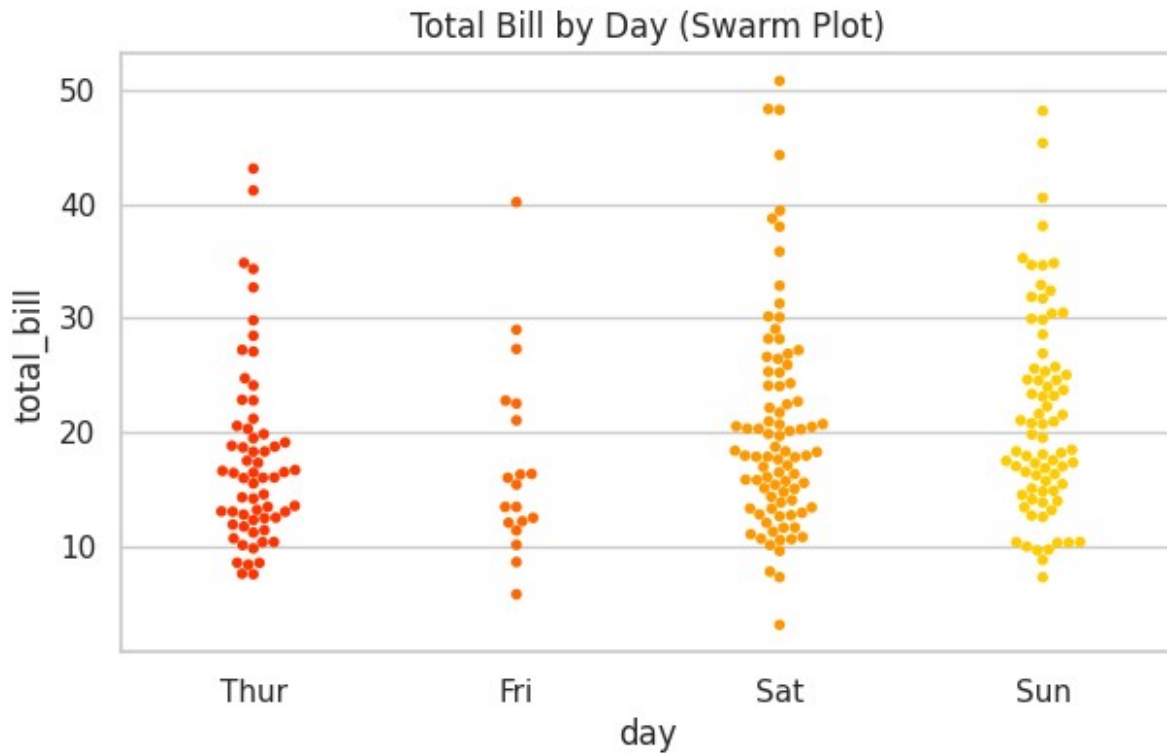
```
plt.figure(figsize=(7, 4))
sns.kdeplot(data=titanic, x="fare", hue="survived", fill=True,
common_norm=False, palette="Set1")
plt.title("Fare Distribution by Survival (KDE Plot)")
plt.xlim(0, 300)
plt.show()
```



```
plt.figure(figsize=(7, 4))
sns.stripplot(x="day", y="total_bill", data=tips, hue="day",
jitter=True, palette="cool", legend=False)
plt.title("Total Bill by Day (Strip Plot)")
plt.show()
```

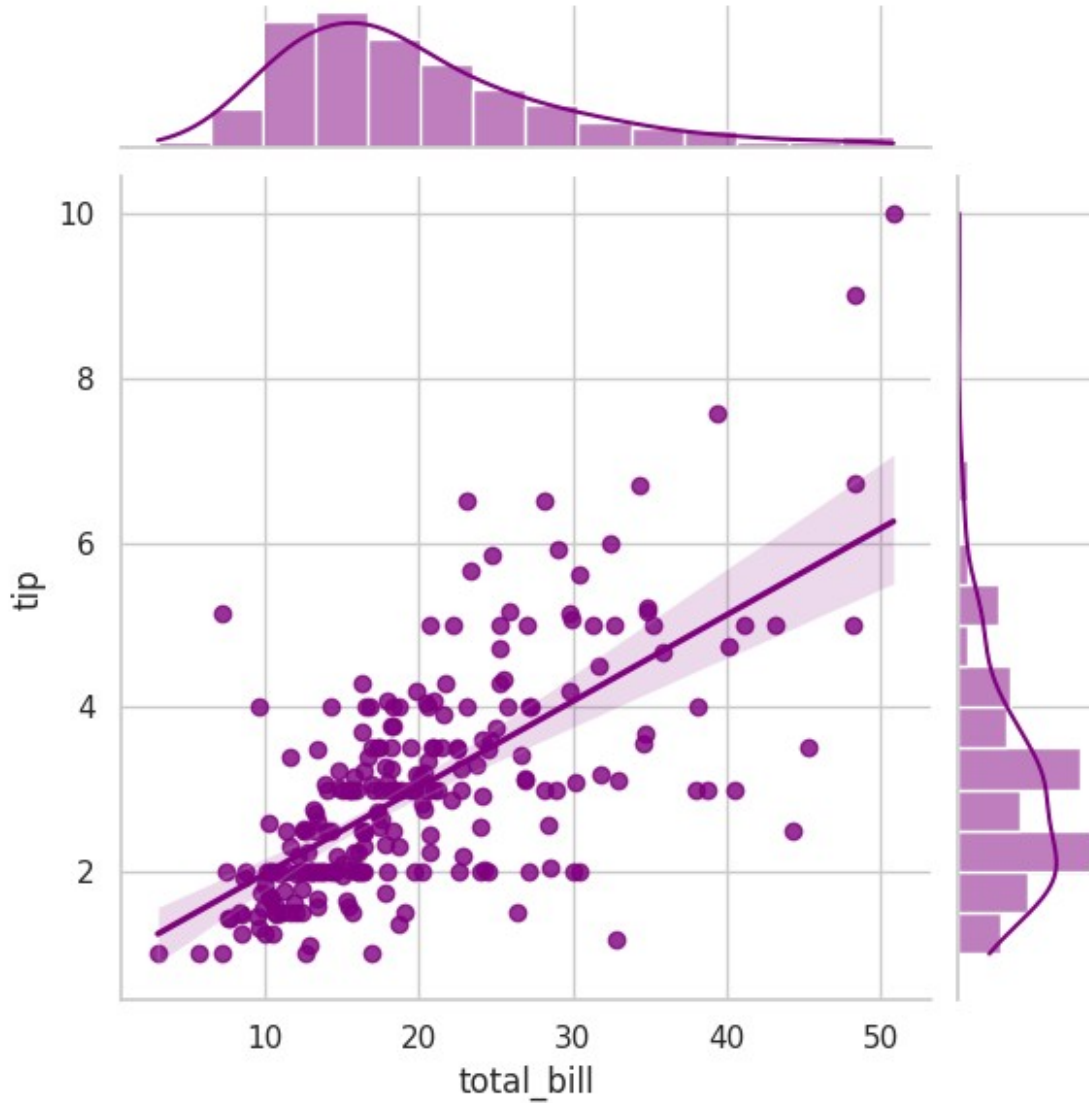


```
plt.figure(figsize=(7, 4))
sns.swarmplot(x="day", y="total_bill", data=tips, hue="day",
              palette="autumn", size=4, legend=False)
plt.title("Total Bill by Day (Swarm Plot)")
plt.show()
```

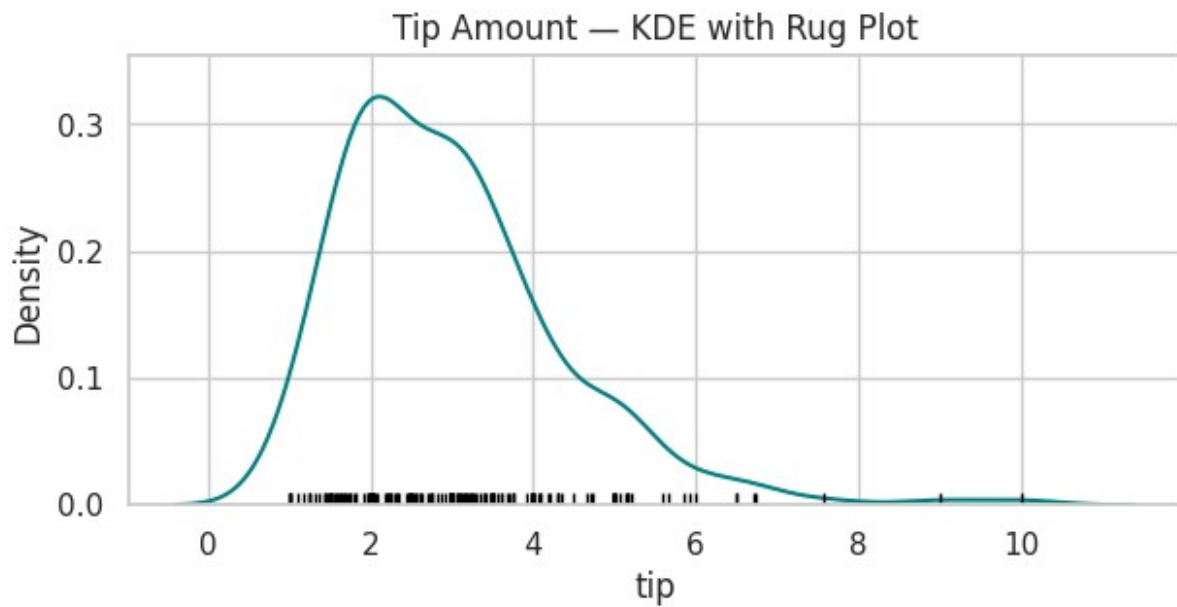


```
sns.jointplot(x="total_bill", y="tip", data=tips, kind="reg",  
color="purple", height=6)  
plt.suptitle("Total Bill vs Tip (Jointplot with Regression Line)",  
y=1.02)  
plt.show()
```

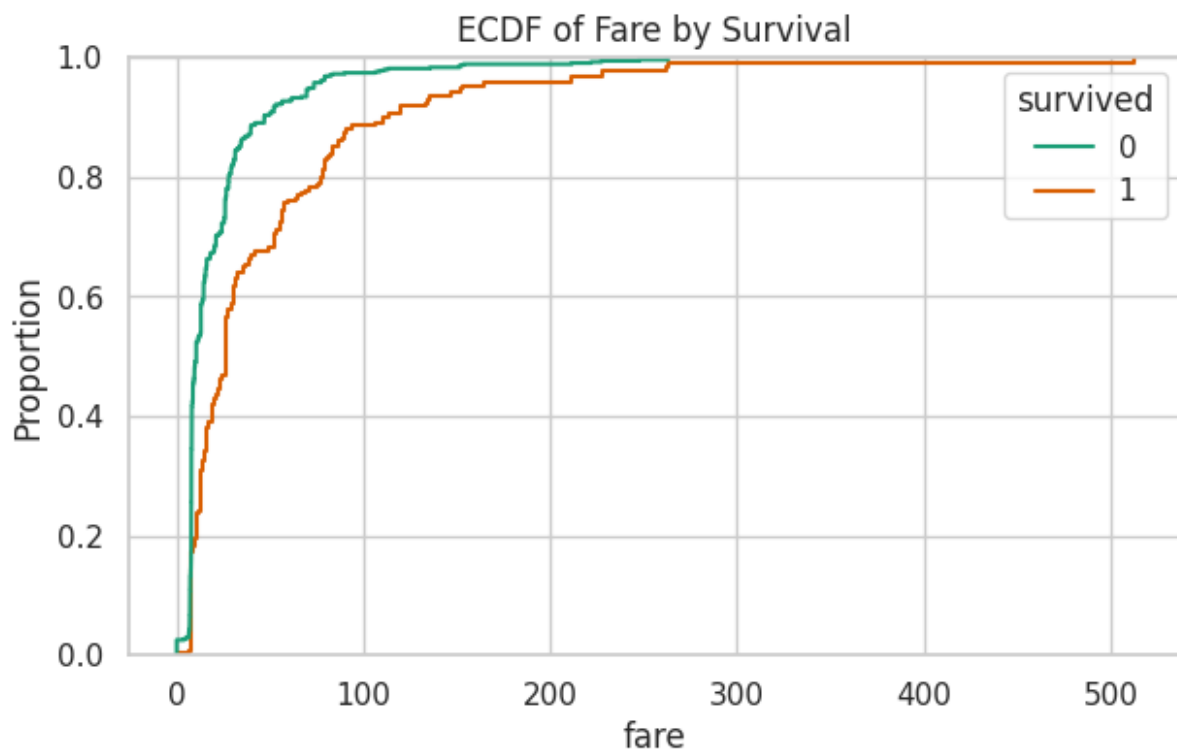
Total Bill vs Tip (Jointplot with Regression Line)



```
plt.figure(figsize=(7, 3))
sns.kdeplot(tips["tip"], color="teal")
sns.rugplot(tips["tip"], color="black")
plt.title("Tip Amount – KDE with Rug Plot")
plt.show()
```



```
plt.figure(figsize=(7, 4))  
sns.ecdfplot(data=titanic, x="fare", hue="survived", palette="Dark2")  
plt.title("ECDF of Fare by Survival")  
plt.show()
```



```
# groupby() + single aggregation
```

```
tips.groupby("day", observed=True)["total_bill"].mean()
```

```
day
Thur    17.682742
Fri     17.151579
Sat     20.441379
Sun     21.410000
Name: total_bill, dtype: float64
```

```
# groupby() + agg() with multiple statistics at once
```

```
tips.groupby("day", observed=True)["total_bill"].agg(["mean",
"median", "std", "count"])
```

```
{"summary":{"\n  \"name\": \"tips\",\n  \"rows\": 4,\n  \"fields\": [\n    {\n      \"column\": \"day\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          \"Fri\",\n          \"Sun\",\n          \"Thur\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"mean\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 2.075247561448747,\n        \"min\": 17.15157894736842,\n        \"max\": 21.41,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          17.15157894736842,\n          21.41,\n          17.682741935483868\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"median\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1.9316033926939205,\n        \"min\": 15.38,\n        \"max\": 19.63,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          15.38,\n          19.63,\n          16.2\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"std\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.6890603661686633,\n        \"min\": 7.886170333087847,\n        \"max\": 9.480418641086723,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          8.302659723217145,\n          8.83212182886989,\n          7.886170333087847\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"count\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 29,\n        \"min\": 19,\n        \"max\": 87,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          19,\n          76,\n          62\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  }}, \"type\": \"dataframe\"}
```

```
# groupby() with multiple columns
```

```
tips.groupby(["day", "time", observed=True])["tip"].mean().unstack()
```

```
{"summary":{"\n  \"name\": \"tips\",\n  \"rows\": 4,\n  \"fields\": [\n    {\n      \"column\": \"day\",\n      \"properties\": {\n
```

```

\dtype\": \"category\", \n          \num_unique_values\": 4, \n
\samples\": [ \n          \"Fri\", \n          \"Sun\", \n
\Thur\" \n          ], \n          \semantic_type\": \"\", \n
\description\": \"\" \n          } \n          }, \n          { \n          \column\":
\Lunch\", \n          \properties\": { \n          \dtype\": \"number\", \n
\std\": 0.27212847155125364, \n          \min\": 2.382857142857143, \n
\max\": 2.767704918032787, \n          \num_unique_values\": 2, \n
\samples\": [ \n          2.382857142857143, \n
2.767704918032787 \n          ], \n          \semantic_type\": \"\", \n
\description\": \"\" \n          } \n          }, \n          { \n          \column\":
\Dinner\", \n          \properties\": { \n          \dtype\": \"number\", \n
\std\": 0.14128172638789066, \n          \min\": 2.94, \n
\max\": 3.2551315789473683, \n          \num_unique_values\": 4, \n
\samples\": [ \n          2.94, \n          3.2551315789473683 \n
], \n          \semantic_type\": \"\", \n          \description\": \"\" \n
} \n          } \n          ], \n          \"type\": \"dataframe\"}

```

*# pivot\_table() – similar to groupby, but in a spreadsheet-like format*  
pd.pivot\_table(tips, values="total\_bill", index="day", columns="time",  
aggfunc="mean", observed=True)

```

{"summary": "{ \n \name\": \"          aggfunc=\\\"mean\\\",
observed=True) \n \rows\": 4, \n \fields\": [ \n          { \n
\column\": \"day\", \n          \properties\": { \n          \dtype\":
\category\", \n          \num_unique_values\": 4, \n          \samples\":
[ \n          \"Fri\", \n          \"Sun\", \n          \"Thur\" \n
], \n          \semantic_type\": \"\", \n          \description\": \"\" \n
} \n          }, \n          { \n          \column\": \"Lunch\", \n          \properties\":
{ \n          \dtype\": \"number\", \n          \std\":
3.4075757303301963, \n          \min\": 12.845714285714285, \n
\max\": 17.664754098360653, \n          \num_unique_values\": 2, \n
\samples\": [ \n          12.845714285714285, \n
17.664754098360653 \n          ], \n          \semantic_type\": \"\", \n
\description\": \"\" \n          } \n          }, \n          { \n          \column\":
\Dinner\", \n          \properties\": { \n          \dtype\": \"number\", \n
\std\": 1.1199622396103324, \n          \min\": 18.78, \n
\max\": 21.41, \n          \num_unique_values\": 4, \n
\samples\": [ \n          19.663333333333334, \n          21.41 \n
], \n          \semantic_type\": \"\", \n          \description\": \"\" \n
} \n          } \n          ], \n          \"type\": \"dataframe\"}

```

*# crosstab() – frequency table between two categorical columns*  
pd.crosstab(titanic["pclass"], titanic["survived"], margins=True)

```

{"summary": "{ \n \name\": \"pd\", \n \rows\": 4, \n \fields\": [ \n
{ \n          \column\": \"pclass\", \n          \properties\": { \n
\dtype\": \"string\", \n          \num_unique_values\": 4, \n
\samples\": [ \n          2, \n          \"All\", \n          1 \n
], \n          \semantic_type\": \"\", \n          \description\": \"\" \n
} \n          }, \n          { \n          \column\": 0, \n          \properties\": { \n

```

```

\"dtype\": \"number\", \"std\": 226, \"min\": 80, \"
\"max\": 549, \"num_unique_values\": 4, \"samples\":
[ 97, 549, 80 ], \"
\"semantic_type\": \"\", \"description\": \"\"
n }, { \"column\": 1, \"properties\": { \"
\"dtype\": \"number\", \"std\": 115, \"min\": 87, \"
\"max\": 342, \"num_unique_values\": 4, \"samples\":
[ 87, 342, 136 ], \"
\"semantic_type\": \"\", \"description\": \"\"
n }, { \"column\": \"All\", \"properties\": { \"
\"dtype\": \"number\", \"std\": 327, \"min\": 184, \"
\"max\": 891, \"num_unique_values\": 4, \"samples\":
[ 184, 891, 216 ], \"
\"semantic_type\": \"\", \"description\": \"\"
n } ] }\", \"type\": \"dataframe\"}

```

*# Start fresh with a clean copy*

```

clean_df = titanic.copy()
print("Starting shape:", clean_df.shape)
clean_df.head()

```

Starting shape: (891, 15)

```

{ \"summary\": { \"name\": \"clean_df\", \"rows\": 891, \"
\"fields\": [ { \"column\": \"survived\", \"
\"properties\": { \"dtype\": \"number\", \"std\":
0, \"min\": 0, \"max\": 1, \"
\"num_unique_values\": 2, \"samples\": [ 1,
0 ], \"semantic_type\": \"\", \"
\"description\": \"\" } }, { \"column\":
\"pclass\", \"properties\": { \"dtype\": \"number\", \"
\"std\": 0, \"min\": 1, \"max\": 3, \"
\"num_unique_values\": 3, \"samples\": [ 3,
1 ], \"semantic_type\": \"\", \"
\"description\": \"\" } }, { \"column\":
\"sex\", \"properties\": { \"dtype\": \"category\", \"
\"num_unique_values\": 2, \"samples\": [ \"
female\", \"male\" ], \"
\"semantic_type\": \"\", \"description\": \"\"
n }, { \"column\": \"age\", \"properties\": { \"
\"dtype\": \"number\", \"std\": 14.526497332334044, \"
\"min\": 0.42, \"max\": 80.0, \"num_unique_values\":
88, \"samples\": [ 0.75, 22.0
n ], \"semantic_type\": \"\", \"
\"description\": \"\" } }, { \"column\":
\"sibsp\", \"properties\": { \"dtype\": \"number\", \"
\"std\": 1, \"min\": 0, \"max\": 8, \"
\"num_unique_values\": 7, \"samples\": [ 1,
0 ], \"semantic_type\": \"\", \"
\"description\": \"\" } }, { \"column\":

```

```

\"parch\",\\n      \\\"properties\\\": {\\n          \\\"dtype\\\": \\\"number\\\",\\n
\\\"std\\\": 0,\\n          \\\"min\\\": 0,\\n          \\\"max\\\": 6,\\n
\\\"num_unique_values\\\": 7,\\n          \\\"samples\\\": [\\n              0,\\n
1\\n          ],\\n          \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n      }\\n  },\\n  {\\n      \\\"column\\\":
\\\"fare\\\",\\n      \\\"properties\\\": {\\n          \\\"dtype\\\": \\\"number\\\",\\n
\\\"std\\\": 49.693428597180905,\\n          \\\"min\\\": 0.0,\\n          \\\"max\\\":
512.3292,\\n          \\\"num_unique_values\\\": 248,\\n          \\\"samples\\\":
[\\n              11.2417,\\n              51.8625\\n          ],\\n
\\\"semantic_type\\\": \\\"\\\",\\n          \\\"description\\\": \\\"\\\"\\n      }\\n
n  },\\n  {\\n      \\\"column\\\": \\\"embarked\\\",\\n      \\\"properties\\\":
{\\n          \\\"dtype\\\": \\\"category\\\",\\n          \\\"num_unique_values\\\":
3,\\n          \\\"samples\\\": [\\n              \\\"S\\\",\\n              \\\"C\\\"\\n
],\\n          \\\"semantic_type\\\": \\\"\\\",\\n          \\\"description\\\": \\\"\\\"\\n
}\\n  },\\n  {\\n      \\\"column\\\": \\\"class\\\",\\n      \\\"properties\\\":
{\\n          \\\"dtype\\\": \\\"category\\\",\\n          \\\"num_unique_values\\\":
3,\\n          \\\"samples\\\": [\\n              \\\"Third\\\",\\n              \\\"First\\\"\\n
n          ],\\n          \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n      }\\n  },\\n  {\\n      \\\"column\\\":
\\\"who\\\",\\n      \\\"properties\\\": {\\n          \\\"dtype\\\": \\\"category\\\",\\n
\\\"num_unique_values\\\": 3,\\n          \\\"samples\\\": [\\n              \\\"man\\\",\\n
n              \\\"woman\\\"\\n          ],\\n          \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n      }\\n  },\\n  {\\n      \\\"column\\\":
\\\"adult_male\\\",\\n      \\\"properties\\\": {\\n          \\\"dtype\\\":
\\\"boolean\\\",\\n          \\\"num_unique_values\\\": 2,\\n          \\\"samples\\\":
[\\n              false,\\n              true\\n          ],\\n
\\\"semantic_type\\\": \\\"\\\",\\n          \\\"description\\\": \\\"\\\"\\n      }\\n
n  },\\n  {\\n      \\\"column\\\": \\\"deck\\\",\\n      \\\"properties\\\": {\\n
\\\"dtype\\\": \\\"category\\\",\\n          \\\"num_unique_values\\\": 7,\\n
\\\"samples\\\": [\\n              \\\"C\\\",\\n              \\\"E\\\"\\n          ],\\n
\\\"semantic_type\\\": \\\"\\\",\\n          \\\"description\\\": \\\"\\\"\\n      }\\n
n  },\\n  {\\n      \\\"column\\\": \\\"embark_town\\\",\\n      \\\"properties\\\":
{\\n          \\\"dtype\\\": \\\"category\\\",\\n
\\\"num_unique_values\\\": 3,\\n          \\\"samples\\\": [\\n
\\\"Southampton\\\",\\n          \\\"Cherbourg\\\"\\n          ],\\n
\\\"semantic_type\\\": \\\"\\\",\\n          \\\"description\\\": \\\"\\\"\\n      }\\n
n  },\\n  {\\n      \\\"column\\\": \\\"alive\\\",\\n      \\\"properties\\\": {\\n
\\\"dtype\\\": \\\"category\\\",\\n          \\\"num_unique_values\\\": 2,\\n
\\\"samples\\\": [\\n              \\\"yes\\\",\\n              \\\"no\\\"\\n          ],\\n
\\\"semantic_type\\\": \\\"\\\",\\n          \\\"description\\\": \\\"\\\"\\n      }\\n
n  },\\n  {\\n      \\\"column\\\": \\\"alone\\\",\\n      \\\"properties\\\": {\\n
\\\"dtype\\\": \\\"boolean\\\",\\n          \\\"num_unique_values\\\": 2,\\n
\\\"samples\\\": [\\n              true,\\n              false\\n          ],\\n
\\\"semantic_type\\\": \\\"\\\",\\n          \\\"description\\\": \\\"\\\"\\n      }\\n
n  }\\n  ]\\n}\\",\"type\":\"dataframe\",\"variable_name\":\"clean_df\"}

```

```

# 'age' -> fill with median (skewed, has outliers)
clean_df["age"] = clean_df["age"].fillna(clean_df["age"].median())

```

```

# 'embarked' -> fill with mode (categorical, very few missing)

```

```

clean_df["embarked"] =
clean_df["embarked"].fillna(clean_df["embarked"].mode()[0])

# 'deck' -> too many missing (~77%) -> drop the column entirely
clean_df = clean_df.drop(columns=["deck"])

# drop rows still missing anything critical (safety net)
clean_df = clean_df.dropna(subset=["embark_town"])

print("Missing values remaining:\n", clean_df.isnull().sum()
[clean_df.isnull().sum() > 0])

Missing values remaining:
Series([], dtype: int64)

before = clean_df.shape[0]
clean_df = clean_df.drop_duplicates()
after = clean_df.shape[0]
print(f"Removed {before - after} duplicate rows. New shape:
{clean_df.shape}")

Removed 116 duplicate rows. New shape: (773, 14)

Q1, Q3 = clean_df["fare"].quantile([0.25, 0.75])
IQR = Q3 - Q1
lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR

n_outliers = clean_df[(clean_df["fare"] < lower) | (clean_df["fare"] >
upper)].shape[0]
print(f"Fare outliers detected: {n_outliers}")

# Cap (winsorize) rather than delete, to avoid losing too many rows
clean_df["fare"] = clean_df["fare"].clip(lower=lower, upper=upper)
print("Fare range after capping:", clean_df["fare"].min(), "-",
clean_df["fare"].max())

Fare outliers detected: 100
Fare range after capping: 0.0 - 71.675

# Drop redundant / high-cardinality / identifier-like columns first
clean_df = clean_df.drop(columns=["who", "adult_male", "alive",
"class", "embark_town"])

# One-Hot Encode nominal columns
clean_df = pd.get_dummies(clean_df, columns=["sex", "embarked"],
drop_first=True)

clean_df.head()

{"summary": "{\n  \"name\": \"clean_df\", \n  \"rows\": 773, \n
\"fields\": [\n    {\n      \"column\": \"survived\", \n

```

```

\"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 0, \n          \"min\": 0, \n          \"max\": 1, \n          \"num_unique_values\": 2, \n          \"samples\": [\n            0, \n            1, \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          \"column\": \"pclass\", \n          \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 0, \n            \"min\": 1, \n            \"max\": 3, \n            \"num_unique_values\": 3, \n            \"samples\": [\n              1, \n              3, \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\", \n            \"column\": \"age\", \n            \"properties\": {\n              \"dtype\": \"number\", \n              \"std\": 13.731264221326203, \n              \"min\": 0.42, \n              \"max\": 80.0, \n              \"num_unique_values\": 88, \n              \"samples\": [\n                0.75, \n                22.0, \n              ], \n              \"semantic_type\": \"\", \n              \"description\": \"\", \n              \"column\": \"sibsp\", \n              \"properties\": {\n                \"dtype\": \"number\", \n                \"std\": 0, \n                \"min\": 0, \n                \"max\": 8, \n                \"num_unique_values\": 7, \n                \"samples\": [\n                  0, \n                  1, \n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\", \n                \"column\": \"parch\", \n                \"properties\": {\n                  \"dtype\": \"number\", \n                  \"std\": 0, \n                  \"min\": 0, \n                  \"max\": 6, \n                  \"num_unique_values\": 7, \n                  \"samples\": [\n                    0, \n                    1, \n                  ], \n                  \"semantic_type\": \"\", \n                  \"description\": \"\", \n                  \"column\": \"fare\", \n                  \"properties\": {\n                    \"dtype\": \"number\", \n                    \"std\": 22.31504568746936, \n                    \"min\": 0.0, \n                    \"max\": 71.675, \n                    \"num_unique_values\": 209, \n                    \"samples\": [\n                      7.2292, \n                      30.0, \n                    ], \n                    \"semantic_type\": \"\", \n                    \"description\": \"\", \n                    \"column\": \"alone\", \n                    \"properties\": {\n                      \"dtype\": \"boolean\", \n                      \"num_unique_values\": 2, \n                      \"samples\": [\n                        true, \n                        false, \n                      ], \n                      \"semantic_type\": \"\", \n                      \"description\": \"\", \n                      \"column\": \"sex_male\", \n                      \"properties\": {\n                        \"dtype\": \"boolean\", \n                        \"num_unique_values\": 2, \n                        \"samples\": [\n                          false, \n                          true, \n                        ], \n                        \"semantic_type\": \"\", \n                        \"description\": \"\", \n                        \"column\": \"embarked_Q\", \n                        \"properties\": {\n                          \"dtype\": \"boolean\", \n                          \"num_unique_values\": 2, \n                          \"samples\": [\n                            true, \n                            false, \n                          ], \n                          \"semantic_type\": \"\", \n                          \"description\": \"\", \n                          \"column\": \"embarked_S\", \n                          \"properties\": {\n                            \"dtype\": \"boolean\", \n                            \"num_unique_values\": 2, \n                            \"samples\": [\n                              false, \n                              true, \n                            ], \n                            \"semantic_type\": \"\", \n                            \"description\": \"\", \n                            \"column\": \"\", \n                          }, \n                        }, \n                      }, \n                    }, \n                  }, \n                }, \n              }, \n            }, \n          }, \n        ], \n      }, \n    ], \n  }, \n  \"type\": \"dataframe\", \"variable_name\": \"clean_df\"}

```

```

num_cols = ["age", "fare"]
scaler = StandardScaler()
clean_df[num_cols] = scaler.fit_transform(clean_df[num_cols])

clean_df[num_cols].describe().round(2)

{"summary": "{\n  \"name\": \"clean_df[num_cols]\", \n  \"rows\": 8, \n  \"fields\": [\n    {\n      \"column\": \"age\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 273.18554473104905, \n        \"min\": -2.12, \n        \"max\": 773.0, \n        \"num_unique_values\": 8, \n        \"samples\": [\n          0.0, \n          -0.11, \n          773.0\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"fare\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 273.25275083898316, \n        \"min\": -1.18, \n        \"max\": 773.0, \n        \"num_unique_values\": 8, \n        \"samples\": [\n          -0.0, \n          -0.46, \n          773.0\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }\n  ], \n  \"type\": \"dataframe\"}"}

print("Final cleaned shape:", clean_df.shape)
print("\nRemaining missing values:", clean_df.isnull().sum().sum())
clean_df.head()

```

Final cleaned shape: (773, 10)

Remaining missing values: 0

```

{"summary": "{\n  \"name\": \"clean_df\", \n  \"rows\": 773, \n  \"fields\": [\n    {\n      \"column\": \"survived\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0, \n        \"min\": 0, \n        \"max\": 1, \n        \"num_unique_values\": 2, \n        \"samples\": [\n          1, \n          0\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"pclass\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0, \n        \"min\": 1, \n        \"max\": 3, \n        \"num_unique_values\": 3, \n        \"samples\": [\n          3, \n          1\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"age\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 1.0006474587923397, \n        \"min\": -2.121232469397667, \n        \"max\": 3.6780532691301464, \n        \"num_unique_values\": 88, \n        \"samples\": [\n          -2.0971841621129954, \n          -0.5486189202970109\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"sibsp\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0, \n        \"min\": 0, \n        \"max\": 8, \n        \"num_unique_values\": 7, \n        \"samples\": [\n          1, \n          1\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }\n  ], \n  \"type\": \"dataframe\"}"}

```

```

0\n      ],\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n    },\n    {\n      \"column\":\n      \"parch\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n        \"max\": 6,\n        \"num_unique_values\": 7,\n        \"samples\": [\n          0,\n          1\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      {\n        \"column\":\n        \"fare\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 1.0006474587923386,\n          \"min\": -1.1769307400804676,\n          \"max\": 2.037107340478555,\n          \"num_unique_values\": 209,\n          \"samples\": [\n            -0.85276019118647,\n            0.1683241244716172\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        {\n          \"column\":\n          \"alone\",\n          \"properties\": {\n            \"dtype\": \"boolean\",\n            \"num_unique_values\": 2,\n            \"samples\": [\n              true,\n              false\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          {\n            \"column\":\n            \"sex_male\",\n            \"properties\": {\n              \"dtype\":\n              \"boolean\",\n              \"num_unique_values\": 2,\n              \"samples\":\n              [\n                false,\n                true\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            }\n          }\n        },\n        {\n          \"column\": \"embarked_Q\",\n          \"properties\": {\n            \"dtype\": \"boolean\",\n            \"num_unique_values\": 2,\n            \"samples\": [\n              true,\n              false\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          {\n            \"column\":\n            \"embarked_S\",\n            \"properties\": {\n              \"dtype\":\n              \"boolean\",\n              \"num_unique_values\": 2,\n              \"samples\":\n              [\n                false,\n                true\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            }\n          }\n        }\n      }\n    }\n  ],\n  \"type\": \"dataframe\", \"variable_name\": \"clean_df\"}

```

```

# Save the final cleaned dataset to CSV
clean_df.to_csv("titanic_cleaned.csv", index=False)
print(" Saved as 'titanic_cleaned.csv'")

```

Saved as 'titanic\_cleaned.csv'

*# STEP 1 (example, already solved): First look at the data*

```

print(tips.shape)
tips.head()

```

(244, 7)

```

{"summary": "{\n  \"name\": \"tips\",\n  \"rows\": 244,\n  \"fields\":\n  [\n    {\n      \"column\": \"total_bill\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 8.902411954856856,\n        \"min\": 3.07,\n        \"max\": 50.81,\n        \"num_unique_values\": 229,\n        \"samples\": [\n          22.12,\n          20.23,\n          14.78\n        ],\n      }\n    }\n  ]\n}

```

```

{"semantic_type": "\n", "description": "\n", "column": "tip", "properties": {"dtype": "number", "std": 1.3836381890011826, "min": 1.0, "max": 10.0, "num_unique_values": 123, "samples": [3.35, 1.5, 6.73]}, "semantic_type": "\n", "description": "\n", "column": "sex", "properties": {"dtype": "category", "num_unique_values": 2, "samples": ["Male", "Female"]}, "semantic_type": "\n", "description": "\n", "column": "smoker", "properties": {"dtype": "category", "num_unique_values": 2, "samples": ["Yes", "No"]}, "semantic_type": "\n", "description": "\n", "column": "day", "properties": {"dtype": "category", "num_unique_values": 4, "samples": ["Sat", "Fri"]}, "semantic_type": "\n", "description": "\n", "column": "time", "properties": {"dtype": "category", "num_unique_values": 2, "samples": ["Lunch", "Dinner"]}, "semantic_type": "\n", "description": "\n", "column": "size", "properties": {"dtype": "number", "std": 0, "min": 1, "max": 6, "num_unique_values": 6, "samples": [2, 3]}], "type": "dataframe", "variable_name": "tips"}

```

```

# STEP 2: Check for missing values
# TODO: use isnull().sum() on the tips dataset
tips.isnull().sum()

```

```

total_bill    0
tip           0
sex           0
smoker        0
day           0
time          0
size          0
dtype: int64

```

```

# STEP 3: Check for duplicate rows
# TODO: use duplicated().sum()
tips.duplicated().sum()

np.int64(1)

```

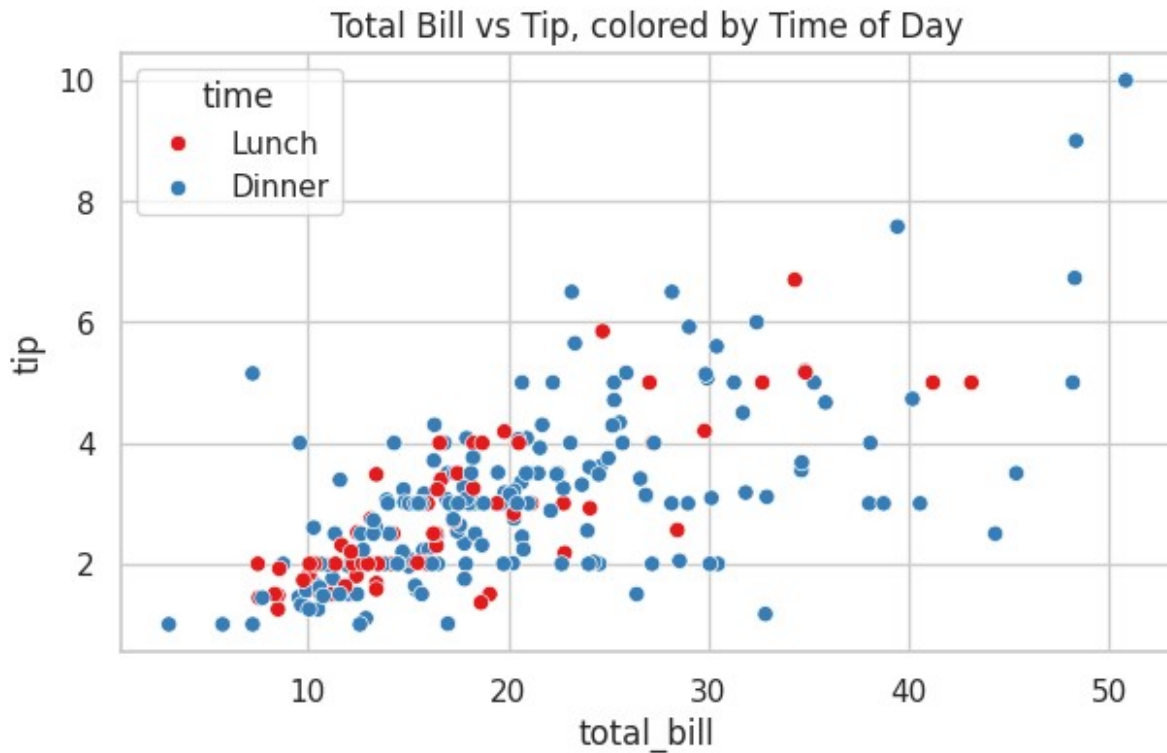
```

# STEP 4: Get summary statistics
# TODO: use describe()
tips.describe()

{"summary": "{\n  \"name\": \"tips\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"total_bill\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 80.57800601480773,\n        \"min\": 3.07,\n        \"max\": 244.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          19.78594262295082,\n          17.795,\n          244.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"tip\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 85.10953159599222,\n        \"min\": 1.0,\n        \"max\": 244.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          2.99827868852459,\n          2.9,\n          244.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"size\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 85.39686221774684,\n        \"min\": 0.9510998047322332,\n        \"max\": 244.0,\n        \"num_unique_values\": 7,\n        \"samples\": [\n          244.0,\n          2.569672131147541,\n          3.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}

# STEP 5: Visualize the relationship between total_bill and tip
# TODO: create a scatter plot, colored by 'time'
plt.figure(figsize=(7, 4))
sns.scatterplot(x="total_bill", y="tip", hue="time", data=tips,
palette="Set1")
plt.title("Total Bill vs Tip, colored by Time of Day")
plt.show()

```



*# Exercise 1: Load Iris and print the first five rows*

*# **TODO:** complete this*

```
iris_ex = sns.load_dataset("iris")
```

```
print(iris_ex.head())
```

|   | sepal_length | sepal_width | petal_length | petal_width | species |
|---|--------------|-------------|--------------|-------------|---------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | setosa  |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | setosa  |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | setosa  |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | setosa  |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | setosa  |

*# Exercise 2: Find missing values in the Titanic dataset*

*# **TODO:** complete this*

```
print(titanic.isnull().sum())
```

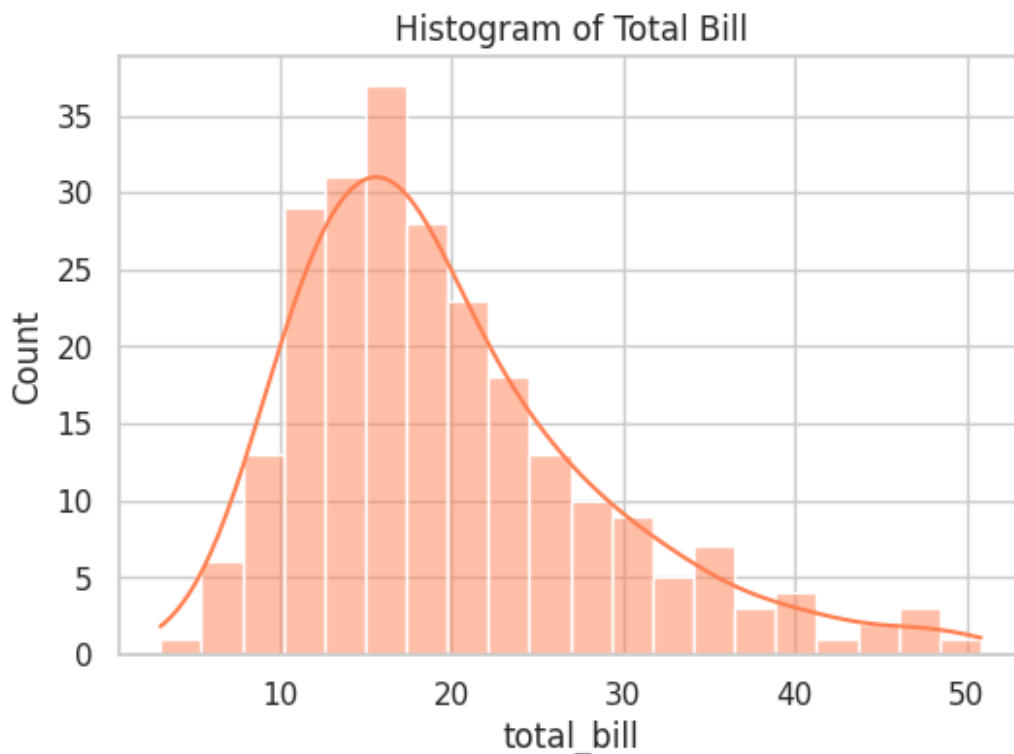
|          |     |
|----------|-----|
| survived | 0   |
| pclass   | 0   |
| sex      | 0   |
| age      | 177 |
| sibsp    | 0   |
| parch    | 0   |
| fare     | 0   |
| embarked | 2   |
| class    | 0   |
| who      | 0   |

```
adult_male    0
deck          688
embark_town    2
alive         0
alone         0
dtype: int64
```

```
# Exercise 3: Remove duplicate rows from the Tips dataset (if any)
# TODO: complete this
tips_no_dup = tips.drop_duplicates()
print("Before:", tips.shape, "After:", tips_no_dup.shape)
```

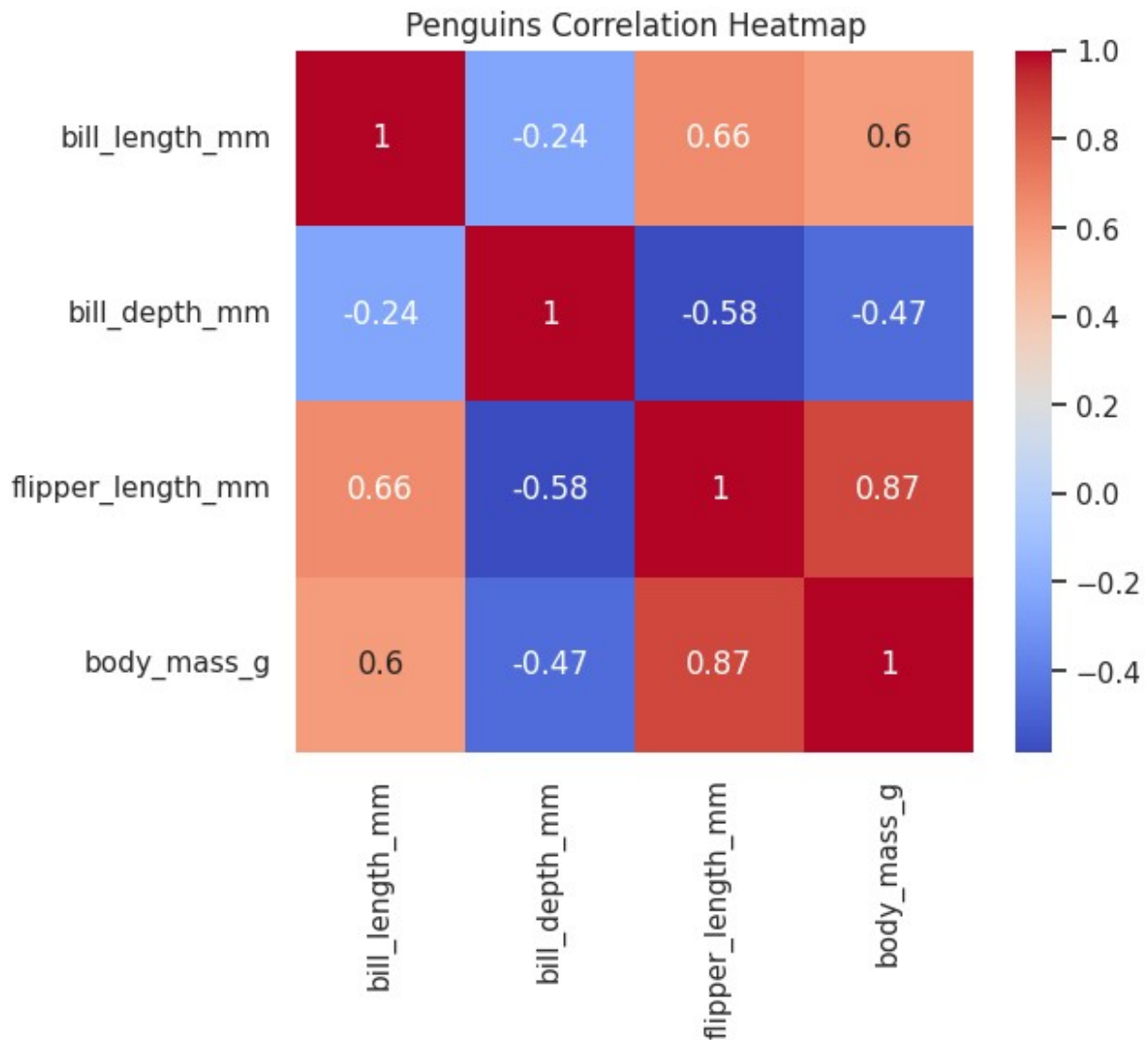
Before: (244, 7) After: (243, 7)

```
# Exercise 4: Plot a histogram of the 'total_bill' column from Tips
# TODO: complete this
plt.figure(figsize=(6, 4))
sns.histplot(tips["total_bill"], bins=20, kde=True, color="coral")
plt.title("Histogram of Total Bill")
plt.show()
```



```
# Exercise 5: Create a correlation heatmap for the Penguins dataset
# TODO: complete this
plt.figure(figsize=(6, 5))
sns.heatmap(penguins.select_dtypes(include=np.number).corr(),
            annot=True, cmap="coolwarm")
```

```
plt.title("Penguins Correlation Heatmap")
plt.show()
```



```
# Your Mini Assignment code goes here
penguins.head()
```

```
{
  "summary": {
    "name": "penguins",
    "rows": 344,
    "fields": [
      {
        "column": "species",
        "properties": {
          "dtype": "category",
          "num_unique_values": 3,
          "samples": [
            "Adelie",
            "Chinstrap",
            "Gentoo"
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "island",
        "properties": {
          "dtype": "category",
          "num_unique_values": 3,
          "samples": [
            "Bordenoise",
            "Dumont",
            "Torgu"
          ],
          "semantic_type": ""
        },
        "description": ""
      }
    ]
  }
}
```

```
[
  {
    "Torgersen": 5.459583713926532,
    "Biscoe": 32.1,
    "Dream": 32.1,
    "description": "bill_length_mm",
    "column": "bill_length_mm",
    "dtype": "number",
    "std": 5.459583713926532,
    "min": 32.1,
    "max": 59.6,
    "num_unique_values": 164,
    "samples": [48.2, 49.8, 45.1]
  },
  {
    "Torgersen": 1.9747931568167816,
    "Biscoe": 18.7,
    "Dream": 18.6,
    "description": "bill_depth_mm",
    "column": "bill_depth_mm",
    "dtype": "number",
    "std": 1.9747931568167816,
    "min": 13.1,
    "max": 21.5,
    "num_unique_values": 80,
    "samples": [16.9, 18.7]
  },
  {
    "Torgersen": 14.061713679356894,
    "Biscoe": 172.0,
    "Dream": 201.0,
    "description": "flipper_length_mm",
    "column": "flipper_length_mm",
    "dtype": "number",
    "std": 14.061713679356894,
    "min": 172.0,
    "max": 231.0,
    "num_unique_values": 55,
    "samples": [180.0, 212.0]
  },
  {
    "Torgersen": 801.9545356980954,
    "Biscoe": 6300.0,
    "Dream": 4350.0,
    "description": "body_mass_g",
    "column": "body_mass_g",
    "dtype": "number",
    "std": 801.9545356980954,
    "min": 2700.0,
    "max": 6300.0,
    "num_unique_values": 94,
    "samples": [4150.0, 3525.0]
  },
  {
    "Torgersen": 1,
    "Biscoe": 1,
    "Dream": 1,
    "description": "sex",
    "column": "sex",
    "dtype": "category",
    "std": 1,
    "min": 1,
    "max": 2,
    "num_unique_values": 2,
    "samples": ["Female", "Male"]
  }
],
{
  "type": "dataframe",
  "variable_name": "penguins"
}
```